



# UTM

UNIVERSITI TEKNOLOGI MALAYSIA

**UNIVERSITI TEKNOLOGI MALAYSIA**

**FACULTY OF COMPUTING**

**SEMESTER II, SESSION 2025/2026**

---

**MINI PROJECT: SMART URBAN MOBILITY ANALYTICS  
PLATFORM (SUMAP)**

**MCST1133-01 (NOSQL DATABASES)**

**LECTURER'S NAME: TS. DR. CHAN WENG HOWE**

---

**GROUP 5 MEMBERS: (UC1)**

| NAME                    | MATRIC NO |
|-------------------------|-----------|
| CHENG YUN               | MCS241014 |
| SIVARAJAN A/L S.ESVARAN | MCS241051 |
| LI HONGLIN              | MCS241031 |
| CUI ZHIWEN              | MCS241040 |

## TABLE OF CONTENTS

|                                                                      |           |
|----------------------------------------------------------------------|-----------|
| <b>1.0 Executive Summary</b>                                         | <b>2</b>  |
| <b>2.0 Use Case Analysis</b>                                         | <b>3</b>  |
| <b>2.1 Problem Description</b>                                       | <b>3</b>  |
| <b>3.0 Data Description &amp; Relationships</b>                      | <b>6</b>  |
| <b>3.1 Entity Descriptions</b>                                       | <b>6</b>  |
| <b>3.3 Data Volume Assumptions</b>                                   | <b>12</b> |
| <b>4.0 Graph Data Model Design [Neo4j]</b>                           | <b>13</b> |
| <b>4.2 Node &amp; Relationship Schema</b>                            | <b>16</b> |
| <b>5.2 Code Walkthrough</b>                                          | <b>20</b> |
| <b>5.3 Sample Data Verification</b>                                  | <b>24</b> |
| <b>6.0 Analytical Queries</b>                                        | <b>26</b> |
| <b>6.1 Analytical Queries (AR-01 to AR-10 Code and Explanations)</b> | <b>26</b> |
| <b>6.2 Query Performance Discussion</b>                              | <b>31</b> |
| <b>7.0 System Reflection &amp; Evaluation</b>                        | <b>31</b> |
| <b>7.1 NoSQL Trade-offs</b>                                          | <b>31</b> |
| <b>7.2 Scalability Considerations</b>                                | <b>32</b> |
| <b>7.3 Comparison (MongoDB vs. Neo4j)</b>                            | <b>33</b> |
| <b>8. Conclusion</b>                                                 | <b>34</b> |
| <b>References</b>                                                    | <b>35</b> |

## 1.0 Executive Summary

The Smart Urban Mobility Analytics Platform (SUMAP) is a hypothetical transport-intelligence system for Nusaville at the city level. It combines information from Bus Rapid Transit (BRT), Light Rail Transit (LRT), shuttles, shared bicycles and e-scooters using transit. It is designed to support data analytics, passenger feedback analysis, an assessment of the carbon footprints of the transport system; and to assist city planners, transport operators and data analysts in monitoring ridership trends, assessing delays and service gap, analysing and monitoring passenger feedback, and checking on the carbon footprint of the transport system.

The SUMAP dataset is very relationship-dominated and that's why Neo4j was chosen as a NoSQL platform for this project. There are 6 different main data structures: passengers, routes, stops, vehicles, journeys and feedback. These are not in isolation. The Passengers travel between stops to perform a journey and provide feedback on its performance, and the journeys begin and end at stops; journey segments are made using vehicles and routes; routes are used by multiple journeys and serve multiple stops; stops can be served by multiple routes; stops can be served directly with another stop by an interchange stop with directed transfer links. This means that the SUMAP data structure has a natural graph shape.

The principle result of this project is that Neo4j is able to capture the actual organisation of an urban mobility network more directly than a conventional relational database. Several SUMAP analytical needs require traversing relationships and not just the filtering of individual records. For instance, the system should be able to check whether particular stops from Central Interchange can be reached within 2 transfer hops, analyse multimodal journey behaviour, assign or group feedback from service users into stops on a particular route, and analyze route-stop coverage. It's much easier to express these requirements in Neo4j, using nodes and relationships, than in a relational database with multiple join tables.

In general, the results of this project indicate that for SUMAP, Neo4j is considered as a viable NoSQL option to use due to its relationship traversal speed, flexible representation of optional relationships, and graph-based data modelling capabilities. It is particularly appropriate for use in a single transport data layer to model relationships between points, interchange transfers, multi-modal journeys and feedback relationships.

## **2.0 Use Case Analysis**

### **2.1 Problem Description**

The current transport system has several data-management issues for the Nusaville City Transport Authority (NCTA). Problem #1 is data fragmentation. Currently, transport data is handled in four separate systems using one for each operator. Thereby, city planners as well as transport analysts lack synergic perspectives on the entire network. Data cleansing, matching and reconciliation is required before meaningful analysis can be done on data from different sources. This leads to delays and by the time a report is ready it may be for an operational time that's already gone.

Specifically, the SUMAP dataset is a complex dataset. The system can be segmented into six primary data categories: passenger, route, stop, vehicle, journey and feedback data, according to the supplementary interview. The only entities seem simple on their own but make the design of the database tricky when it comes to relationships. All passengers may submit numerous feedback records and make multiple trips. Each journey belongs to exactly one passenger and records both an origin stop and a destination stop. These stop references are mandatory for journey records, while some feedback references to journey, route, or stop are optional, and can have one to four segments. Each segment can be linked to a different route / different vehicle. The system therefore, should reflect not only single records but the relationship between those records also.

The technical problem first encountered is the variability in data of travel. A journey can be a one mode journey, e.g. a single mode BRT journey, or a multi modal journey that combines BRT, LRT and a shared bike in order. In the interview, a segment is one trip in one mode of transport, one vehicle, from one location to another. There are different vehicles, routes and modes of transport that operate at different arrival and departure times at different costs in each segment. If segments were implemented in the same way as tables knew their own existence and only made sense when their parent design it, then the model would be more fragmented and less easy to query.

Second: The stop/route network is very interconnected. The relationship of routes and stops is many to many, meaning that a route may include several stops, and a stop may be operated by more than one route. Moreover, there are directed transfer links in interchange stops that can directly link the stops. It is evident from the interview that these are not the "normal" stop to stop sequence in a route. They are rather links of transfer between various modes of transport at the points of interchange and can also contain attributes like the "typical walking time". This suggests that the physical transport network is not just the tabular form, it is essentially of a graph form.

The third structure problem is heterogeneous vehicle telemetry. Various types of transport employ different forms of vehicle telemetry, and the interview provides examples of this. A BRT bus can provide battery percentage, average speed, air conditioning, GPS time stamp and engine temperature. The following report can be sent by an LRT train: Voltage of the pantograph, status of the train doors and signal block ID, and traction mode information. A communal bike can indicate the battery charge level, lock and whether it has been damaged. The structure of these fields is inconsistent, which makes it hard to adopt a single uniformity in the structure of vehicle records.

The fourth structural challenge is mixed nerstructured and optional passenger feedback. Structured fields in the "Feedback" record are those of the category, severity, resolution status, route ID, and stop ID. They also are filled with free-text descriptions and automatically generated keywords. The interview also details that each feedback record should be associated with a passenger but can optionally be associated with a journey, stop, route or journey, stop, route. This will result in both text search needs and flexible relationship requirements that are very hard to effectively fulfil with the present legacy systems.

The issue at hand is not only systems separated, it is the fact that the data itself is variable, is heterogeneous and is relationship intensive. In order to support citizen participation, SUMAP requires a data layer to be redesigned to have the ability to integrate passengers, journeys, routes, stops, vehicles, and feedback. The database should be analytical capable in terms of providing a range of analytical services without undue reconciliation burden and schema rigidity, and minimize the need for complex joins and services, including ridership and delay analyses, multimodal journey behaviours, feedback intelligence, and network connectivity.

## **2.2 Why NoSQL : Justification for Neo4j**

The reason for choosing Neo4j for this project is because it is a graph database that is optimized to store and query highly connected data. A traditional relational database stores data primarily in tables, while Neo4j structures data in terms of nodes, relationships and properties. This model starts from the premise that being able to keep passengers, journeys, routes, stops, vehicles, feedback as separate records is not a feature of SUMAP, but actually analysing the way these records are interconnected throughout the transport network. So far, according to transport database research, graph databases have been integrated into multimodal transport networks and have been applied to conduct relation-oriented queries, multimodal routing, network updates, and so on more flexibly than strict relation oriented schemas (Park & Cheng, 2023).

The first reason to select Neo4j is that the route-stop network is graph-like. The Stop entity properties in the project data include `connected_routes`, and this is the case for the Route entity: properties include `route_id`, `mode`, `total_stops`, and `avg_daily_passengers`. As close relationships between both routes and stops can be recognized in these fields. A route may be able to serve multiple stops and a stop may be served by multiple routes. This is the many-to-many relationship that typically requires two joins (which is very common) and junction tables in a relational database. The route and Stop can be represented as nodes that are linked in a graph database like Neo4j using the real project's fields, such as `route_id` and `connected_routes`. This results into a more realistic model of public transport service, which better suits stop-level analysis as well as route coverage.

The second is interchange connectivity is included in SUMAP. The `is_interchange` field in the Stop entity specifies that a stop is a multi-modal transfer stop. The project relationship description also describes how there are also directed "transfer" relations within the project between some of the interchange stops. Some of these transfer relations may also include properties like walking time. This is because one of the analytical requirements is that all stops reachable from an origin of Central Interchange (STOP-001) in 2 transfer hops. Neo4j is ideal for this need, as traversing "stop-to-stop" relations in the transport network is a natural capability of it. Current works also demonstrate the value of the use of Neo4j and graph-based models to analyze railway and public transport connectivity, and accessibility within public transport systems (Buchwald & Sobczak, 2021; Le Hasif et al., 2022).

The third reason is the complexity of journey data – it varies and has complex relationships. The project data contains the following Journey records: each has a `passenger_id`, an `origin_stop_id`, a `destination_stop_id`, a `transport_mode` (type of transportation used to go from the origin to the destination), and an `n_segments` (a record's length, measured in the number of its segments). A journey is represented in these fields by one of its passengers, who originated it from one stop, to another stop, and may have travelled through one or more transport segments. The field `n_segments`, varying from 1 to 4, reflects the variable nature of journeys. The multimodal journeys section of the project relationship description further specifies that a journey can have multiple route types and modes for multimodal journeys depending on the number of transport portions. So these real fields and references can be mapped to graph relationship between the Passenger, Journey, Stop, Route and Vehicle nodes, which makes Neo4j appropriate. This will give the journey structure the flexibility of not needing to place all journeys into a static table format, with its shape rather determined by the actual project data.

Heterogeneous vehicle telemetry is the fourth reason. The telemetry field is described in the Vehicle entity as variable sensor fields dependent on a Vehicle mode as sub-document. This suggests that BRT

vehicles, LRT vehicles, shuttles, bikes and e-bikes might not have the same type of vehicle telemetry attributes. In addition, fields like mode, fuel\_type, assigned\_route\_id, and current\_status relate vehicle operations to route assignment and service status, respectively, and are included in the Vehicle entity. This change is problematic because it makes it complicated to use one static relational schema for all vehicle records. This follows from the fact that the Vehicle node may contain a variety of properties if they are needed, but is still connected to the Route and Journey data via the real references in the project.

Flexible and optional structure of feedback data is the fifth reason. In the Feedback / Incident Report entity, each feedback record contains an attribute passenger\_id that is used to tie a record to the passenger that submitted it. However, journey\_id, stop\_id and route\_id are optional. A feedback record might include journey, stop or route feedback, but it could also include general feedback that does not include all of the above. These optional references can generate numerous null values in a relational schema and can involve extra joins when analyzing, depending on the current requirements. In Neo4j, the relationships are only existent in the actual graph as a real connection, allowing for a clean and flexible graph model from which to analyse unresolved complaints, route-related issues, and stop related incidents.

### 3.0 Data Description & Relationships

#### 3.1 Entity Descriptions

There are six entities in this scenario, including Passenger, Journey, Route, Stop, Vehicle, and Feedback. Every entity plays a crucial node in the diagram of Neo4j database, which can instantly describe the transportation situation of urban life.

From Table3.1, it is clear to display the basic information of the ‘Passenger’ entity. There are a total 500 rows, with no missing value observed. Most features are objective for data type. The dataset covers the detailed information of a passenger ,including name, age , email and so on.

Table 3.1 The Basic Information of ‘Passenger’ Entity

| Column       | No Null Count | Data Type |
|--------------|---------------|-----------|
| Passenger_id | 500           | objective |
| full_name    | 500           | objective |
| age          | 500           | int64     |
| gender       | 500           | objective |

|                |     |           |
|----------------|-----|-----------|
| email          | 500 | objective |
| phone          | 500 | objective |
| home_zone      | 500 | objective |
| work_zone      | 500 | objective |
| commuter_type  | 500 | objective |
| Registered_at  | 500 | objective |
| preferred_mode | 500 | objective |
| loyalty_tier   | 500 | objective |
| total_journeys | 500 | int64     |
| is_active      | 500 | bool      |

Going through Table 3.2, it presents the basic information of the journey entity. In the ‘Journey’ dataset, it has 17 features, storing the information from every journey of a passenger. It has 3000 rows but there are some missing values in ‘rating’ and ‘promo\_code’. It describes the important information of a journey, including journey destination, journey duration, journey date and so on.

Table 3.2 The Basic Information of ‘Journey’ Entity

| Column              | No Null Count | Data Type |
|---------------------|---------------|-----------|
| journey_id          | 3000          | Objective |
| passenger_id        | 3000          | Objective |
| journey_date        | 3000          | Objective |
| origin_stop_id      | 3000          | Objective |
| destination_stop_id | 3000          | Objective |
| transport_mode      | 3000          | Objective |
| total_fare_myr      | 3000          | float64   |
| fare_category       | 3000          | Objective |
| payment_method      | 3000          | Objective |
| duration_minutes    | 3000          | int64     |
| distance_km         | 3000          | float64   |
| delay_minutes       | 3000          | int64     |



|            |      |           |
|------------|------|-----------|
| status     | 3000 | Objective |
| rating     | 1365 | float64   |
| promo_code | 356  | Objective |
| carbon_g   | 3000 | float64   |
| n_segments | 3000 | int64     |

Table 3.2 demonstrates the basic information of the ‘Route’ dataset, covering route name, operator, distance, start time, end time and so on. There are a total of 21 rows without any missing value. Most features are objective data type from the viewing of the table. It describes the valued information of a route, which can reveal the situation of every route.

Table 3.3 The Basic Information of ‘Route’ Entity

| Column              | No Null Count | Data Type |
|---------------------|---------------|-----------|
| route_id            | 21            | object    |
| route_name          | 21            | object    |
| mode                | 21            | object    |
| operator            | 21            | object    |
| total_shops         | 21            | int64     |
| distance_km         | 21            | float64   |
| frequency_minutes   | 21            | int64     |
| is_express          | 21            | bool      |
| start_time          | 21            | object    |
| end_time            | 21            | object    |
| avg_daily_passenger | 21            | int64     |
| status              | 21            | object    |

Table 3.4 describes the basic information from columns to data type, covering 11 features. There 10 features are without any missing cell, but ‘last\_renovation\_year’ loses some cell value. The data type of most features are the type of object with 120 rows in the viewing of the table. It draws the scenario situation of every stop, including stop name, stop type, zone of stop, latitude and longitude of the stops and so on.

Table 3.4 The Basic Information of ‘Stop’ Entity

| Column                | No Null Count | Data Type |
|-----------------------|---------------|-----------|
| stop_id               | 120           | object    |
| stop_name             | 120           | object    |
| stop_type             | 120           | object    |
| zone                  | 120           | object    |
| latitude              | 120           | float64   |
| longitude             | 120           | float64   |
| is_interchange        | 120           | bool      |
| avg_daily_boarding    | 120           | int64     |
| wheelchair_accessible | 120           | bool      |
| last_renovation_year  | 109           | int64     |
| facilities            | 120           | objective |

Looking at Table 3.5, it demonstrates the basic and detailed information about the vehicle through 22 different features. It totally has 250 rows of every feature, but some of the features have missing values, which are from ‘tel\_ac\_status’ to ‘tel\_is\_damaged’. The features are the properties of every vehicle ,covering the mode of vehicle , model of the vehicle, capacity of the vehicle and so on. Many of the features are the data type of object, some are int64 , float64, and bool.

Table 3.5 The Basic Information of ‘Vehicle’ Entity

| Column            | No Null Count | Data Type |
|-------------------|---------------|-----------|
| vehicle_id        | 250           | object    |
| mode              | 250           | object    |
| model             | 250           | object    |
| manufacture_year  | 250           | int64     |
| capacity          | 250           | int64     |
| fuel_type         | 250           | object    |
| assigned_route_id | 250           | object    |
| current_status    | 250           | object    |

|                          |     |         |
|--------------------------|-----|---------|
| last_service_date        | 250 | object  |
| odometer_km              | 250 | float64 |
| tel_battery_pct          | 250 | float64 |
| tel_avg_speed_kmh        | 150 | float64 |
| tel_ac_status            | 150 | object  |
| tel_last_gps_ping        | 150 | object  |
| tel_engine_temp_c        | 150 | float64 |
| tel_pantograph_voltage_v | 40  | float64 |
| tel_door_status          | 40  | object  |
| tel_signal_block_id      | 40  | object  |
| tel_traction_mode        | 40  | object  |
| tel_lock_status          | 60  | object  |
| tel_last_docked_stop     | 60  | object  |
| tel_is_damaged           | 60  | object  |

Checking the Table 3.6, it covers the information about complaining feedback from passengers , and saves the data through the 12 features. Through this dataset, there are almost half of features which have missing value. The ‘feedback’ presents the id of the passenger, feedback type from passengers, the severity of feedback and so on. All features together can effectively describe the scene of the whole feedback.

Table 3.6 The Basic Information of ‘feedback’ Entity

| Column       | No Null Count | Data Type |
|--------------|---------------|-----------|
| feedback_id  | 300           | object    |
| passenger_id | 300           | object    |
| journey_id   | 214           | object    |
| category     | 300           | object    |
| severity     | 300           | object    |
| description  | 300           | object    |
| submitted_at | 300           | object    |

|                  |     |        |
|------------------|-----|--------|
| stop_id          | 209 | object |
| route_id         | 161 | object |
| is_resolved      | 300 | bool   |
| resolution_notes | 150 | object |
| tags             | 300 | object |

### 3.2 Relationship Diagram

Every Entity can be sorted as a node of rational database, and they are linked by different relationship stamps. Through Figure 3.1, it can be seen that ‘Vehicle’ has two relationships with ‘Route’ and ‘Journey’. By the relationship ‘OPERATE\_ON’, it connects with the node ‘Route’, which indicates ‘Vehicle’ run on which ‘Route’. By the relationship ‘USER\_VEHICLE’, it builds a relationship with the mode ‘Journey’, which suggests every ‘Journey’ uses which ‘Vehicle’ during the operation.

Looking at the node of ‘Stop’, it has five different relationships. By the relationship ‘SERVER’, it links with the ‘Route’ node, which implies that there are stops building on the ‘Route’ . The second relationship is ‘Journey’, which is connected to the ‘Stop’ by relationship ‘STARTED\_AT’ and ‘ENDED\_AT’. It describes ‘Journey’ starts at which stop and end at which stop. The third one is ‘Feedback’, which is linked to ‘Stop’ by the relationship ‘ABOUT\_STOP’, and it said passengers complain which stop. The final one is ‘Stop’ itself, which is linked to ‘Stop’ by the relationship ‘CONNECTED’, and it suggests which stops are assigned on the same lines.

Checking on the node ‘Journey’, it has six relationships, but three of them are mentioned in the above context. It uses relationship ‘USER\_ROUTE’ to link with the ‘Route’ node, which displays every journey facing which route. Through the relationship ‘ABOUT\_JOURNEY’, it connects to the node ‘Feedback’, which presents every feedback that occurs on which journey. By the relationship ‘MADE’, it is linked with the node ‘Passenger’, which reflects that every passenger chooses which journey.

Observing at the node ‘Feedback’, it has four relationships, however, in the above context, it already demonstrated two of them. By the relationship ‘SUBMITTED’, it connects to the node ‘Passenger’, which can display who provides the feedback. Moreover, through the relationship

‘ABOUT\_ROUTE’, it is linked to the node ‘Route’, which presents every feedback that occurs on which route.

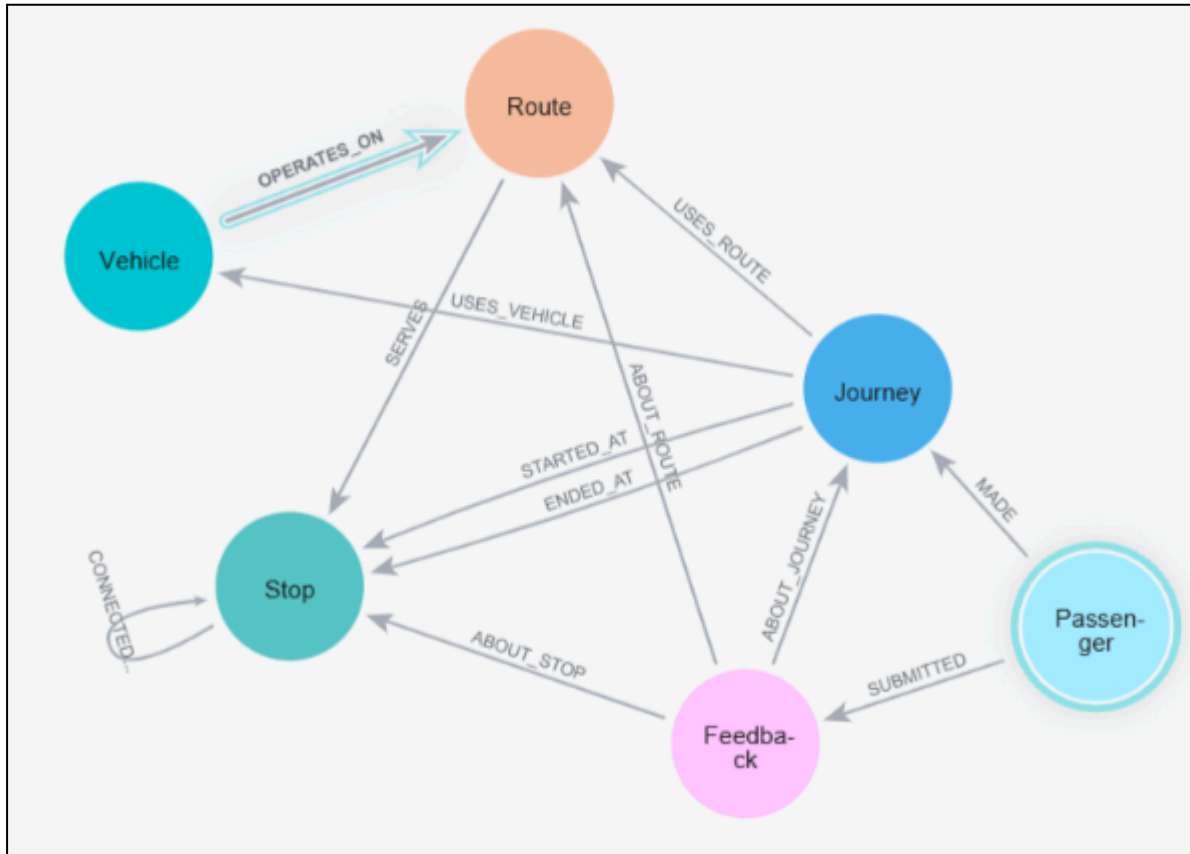


Figure 3.1 The Overview Relationship of All Entities

### 3.3 Data Volume Assumptions

In this mini project, it merges 9 different CSV format files to one database in Neo4j. Going through the cypher query, the figure 3.2 displays the size of the database, and also tells what quantity of the nodes: Feedback, Journey, Passenger, Route, Stop, and Vehicle. From this query, it can find that the quantity of the node ‘Passenger’ is 500, the quantity of the node ‘Journey’ is 3000, the quantity of the node ‘Route’ is 21, the quantity of the node ‘Stop’ is 120, the quantity of the node ‘Feedback’ is 300, and the quantity of the node ‘Vehicle’ is 250.



Figure 3.2 The Overview of Dataset Size

#### 4.0 Graph Data Model Design [Neo4j]

The SUMAP (Smart Urban Mobility Analytics Platform) dataset needed to be modeled using the property graph model in Neo4j, to account for the complex and interconnected nature of urban mobility systems. Neo4j stores entities as nodes and relationships as connections between them, allowing for efficient data traversal and relationship analysis, unlike relational databases that would have multiple table joins to accomplish this.

The graph model has been developed in light of the business requirements given in the case study. The nodes as labels were used to display core entities, and directed relationships were used for depicting interactions between the entities, such as passengers and journeys, which are connected to the nodes. Core entities such as new passengers, journeys, routes, stops, vehicles, and feedback reports were depicted as node labels and directed relationships were used for depicting the interactions between these entities. The design supports complex transportation queries such as route exploration, journey tracking, service connectivity and passenger feedback analysis, and can be implemented efficiently.

Also the graph model provides an analysis support for analysis requirements of the project, which contains network connectivity analysis (AR-09) that makes use of graph traversal. The Neo4j database directly represents the urban mobility ecosystem as a natural database since it stores entities and their relations together.

## 4.1 Property Graph Diagram

Using the property graph approach of Neo4j, SUMAP's transportation system is modelled and represents entities and their relationships in an urban mobility network. There are six major node labels associated with the graph of the property: Passenger, Journey, Route, Stop, Vehicle and Feedback. The nodes contain attributes relevant to the description of the entities corresponding to the nodes while the relationship describe the interaction and dependencies between the attributes stored at the different nodes.

The Passenger node represents registered users' of the transportation system, who are connected with the Journey node via the MADE relationship, which means a passenger may experience multiple journeys. Each node of a Journey has information related to the trip such as travel date, transport mode, fare amount, duration, and the status of the trip. The STARTED\_AT relationship is created to link a journey to the origin stop and the ENDED\_AT relationship is established to link it with the destination stop.

Each Route represents transportation services in the network. Relationships that are important to consider are SUPPORTED\_BY and USES\_ROUTE, which link journeys to routes, and OPERATES\_ON, which links vehicles to routes. The Route and Stop nodes have a SERVES relationship indicating the stops served by a particular route. Furthermore, the CONNECTED\_TO relationship is applied to model the transfer options from stop to stop so that network connectivity and traversal of the routes connecting the stops is possible.

The Feedback node is an indicator of passenger feedback. A SUBMITTED relationship conveys a feedback from a passenger; the relationship ABOUT\_JOURNEY, ABOUT\_STOP or ABOUT\_ROUTE can be used to reference a journey, a stop or a route, respectively, on which feedback was registered. The structure allows for detailed evaluation of the quality of the services provided, the operational performance of the system and the satisfaction of the passengers.

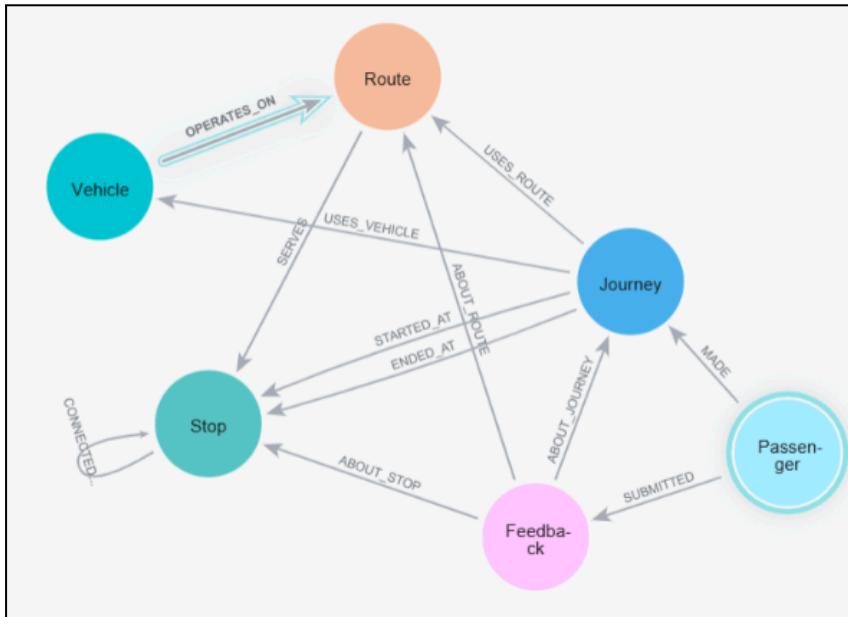


Figure 4.1 The Overview Relationship of All Entities

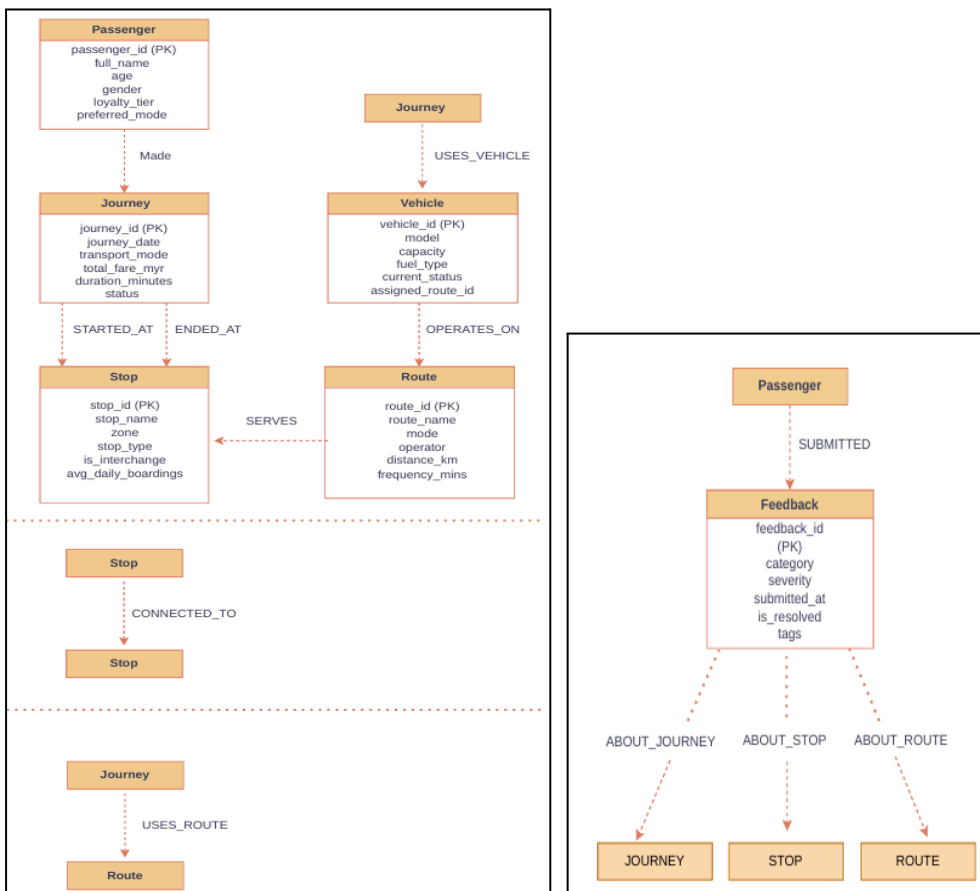


Figure 4.2: Illustrates the conceptual property graph model



## 4.2 Node & Relationship Schema

The SUMAP property graph database is specified via node and relationship schema. Labels were developed, namely at six nodes representing the major entities in the transportation network: Passenger, Journey, Route, Stop, Vehicle, and Feedback. Each node has an identifier as well as a range of properties that can be described uniquely and contain information about how the node would participate when a system is running. Table 4.1 shows a node schema for the graph model.

**Table 4.1: Node Schema**

| Label     | Key Properties                                                                          |
|-----------|-----------------------------------------------------------------------------------------|
| Passenger | passenger_id (PK), full_name, age, gender, loyalty_tier, preferred_mode                 |
| Journey   | journey_id (PK), journey_date, transport_mode, total_fare_myr, duration_minutes, status |
| Route     | route_id (PK), route_name, mode, operator, distance_km, frequency_mins                  |
| Stop      | stop_id (PK), stop_name, zone, stop_type, is_interchange, avg_daily_boardings           |
| Vehicle   | vehicle_id (PK), model, capacity, fuel_type, current_status, assigned_route_id          |
| Feedback  | feedback_id (PK), category, severity, submitted_at, is_resolved, tags                   |

The interactions and connectivity among nodes are represented as relationships. The relationship schema that is implemented in Neo4j is summarized in Table 4.2.

**Table 4.2: Relationship Schema**

| Relationship | Source    | Target  | Properties |
|--------------|-----------|---------|------------|
| MADE         | Passenger | Journey | None       |
| STARTED_AT   | Journey   | Stop    | None       |
| ENDED_AT     | Journey   | Stop    | None       |
| USES_ROUTE   | Journey   | Route   | None       |
| USES_VEHICLE | Journey   | Vehicle | None       |
| OPERATES_ON  | Vehicle   | Route   | None       |
| SERVES       | Route     | Stop    | None       |

|               |           |          |                                                     |
|---------------|-----------|----------|-----------------------------------------------------|
| CONNECTED_TO  | Stop      | Stop     | via_mode, avg_transfer_walk_mins,<br>peak_hour_only |
| SUBMITTED     | Passenger | Feedback | None                                                |
| ABOUT_JOURNEY | Feedback  | Journey  | None                                                |
| ABOUT_STOP    | Feedback  | Stop     | None                                                |
| ABOUT_ROUTE   | Feedback  | Route    | None                                                |

The schema that arises is 6 nodes and 12 relationships. The structure allows performing the efficient traversal of the graph, supports transportation network analysis, exploration of route connectivity, tracking of passenger trips and management of feedback from the transportation services provided within the SUMAP system.

### 4.3 Index & Constraint Design

In order to keep the data valid and to optimize the query performance, structures with constraints and indexes were added to the database neo4j. Constraints can prevent duplicate records by providing uniqueness on you primary identifiers and indexes can speed up certain graph traversal operations and certain record lookups when creating relationships. However, these mechanisms can be particularly crucial when dealing with big transportation-related datasets like SUMAP, where thousands of journeys, stops, routes and vehicles are all contained in the graph database.

**Table 4.3: Constraints Implemented**

| Constraint Name     | Node Label | Property     | Purpose                                        |
|---------------------|------------|--------------|------------------------------------------------|
| passenger_id_unique | Passenger  | passenger_id | Ensures each passenger has a unique identifier |
| journey_id_unique   | Journey    | journey_id   | Prevents duplicate journey records             |
| route_id_unique     | Route      | route_id     | Ensures route uniqueness                       |
| stop_id_unique      | Stop       | stop_id      | Prevents duplicate stop records                |
| vehicle_id_unique   | Vehicle    | vehicle_id   | Ensures each vehicle is uniquely identified    |
| feedback_id_unique  | Feedback   | feedback_id  | Prevents duplicate feedback records            |

The constraints ensure that each node in the SUMAP graph is distinct, and allows for no duplicate nodes to be added to the graph when data is ingested. This is certainly for importing lots of data, and joining entities together.

Identifier fields were also indexed to easily finding them to achieve better performance when traversing the graph and make new relationships easier. Typically, you will find that Neo4j nodes in ingestion cycles will have vastly differing identifiers like `passenger_id`, `journey_id`, `route_id` and `stop_id`. With no indexes, Neo4j would have to do full node scans, and thus run slower. Using indexes, node matching operations are greatly speeded up, which enhances ingestion and query performance.

In sum, constraints and indexes provide greater reliability, consistency, and scalability for the SUMAP graph database.

#### **4.4 Graph Model Discussion**

Using the complex relationships between passengers, journeys, routes, stops, vehicles and feedback records, the SUMAP transportation system was implemented using Neo4j's property graph model. Here, Neo 4j differs from most relational databases. Whereas a relational database would have to use foreign keys to store relationships between nodes/records, relationships are stored in Neo 4j between connected nodes. This facilitates transportation entities and their interactions to be rendered more naturally and efficiently.

Transportation systems are naturally interconnected structures, making them an ideal candidate to use the graph model to represent them. Routes are used in the SUMAP model for more than one stop, vehicles are run on the routes, the passengers are run on a journey and stops are linked to other stops by interchange links. These relationships are all interconnected and working around them in Neo4j can be done relatively efficiently without using an expensive join operation. This means that queries like stop connectivity analysis, route exploration and journey tracking can be performed more effectively in the network.

The graph representation has one of the greatest benefits of supporting representation of interchange networks. The `CONNECTED_TO` relationship directly connects one stop to another, while putting other information about transfers in the system, like average walking time and transfer mode. If a relational database were used, finding the transfer path from one stop to another would involve a number of joins to some of the tables. By contrast, Neo4j offers the ability to "walk the graph

through" connected "stops" directly, which, in many ways, represents an ideal application of transportation network analysis as well as path-finding operations.

Another advantage of the graph modeling is that it enhances the modeling of the activities taken by the passengers and the feedback from the services. The database can easily relate passengers and journeys via relationships like MADE, SUBMITTED, or ABOUT\_JOURNEY, ABOUT\_STOP, or ABOUT\_ROUTE; and relate journeys and feedback records using relationships such as ABOUT\_REFLECTION, ABOUT\_COMMENT, or ABOUT\_SUGGESTION. This space allows analysts to query relationships and understand how passengers behave while analyzing service issues and route performance.

Furthermore, it has flexible extendibility in the future. New node labels, relationships do not involve redesign of the database; other data being added (such as ticketing, maintenance, real-time sensor data) are added as sub-graphs. This easily scalable nature makes Neo4j a good platform for use in smart transportation or mobility analytics applications.

The Neo4j graph model, in general, provides more intuitive and efficient ways to represent the SUMAP transportation ecosystem than one of the relational models. The graph database can facilitate the traversal, analysis of networks, and conducting queries based on relationships, which are essential for comprehending and controlling urban transportation systems.

## **5.0 Data Ingestion**

### **5.1 Ingestion Strategy**

The SUMAP dataset consist of 9 datasets in CSV format (data files), these are passengers, journeys, routes, stops, vehicles, feedback records, stop-route mappings, stop connections, and journey segments. The organized eating process has been done into two stages; creation of nodes, and creation of relations.

In the first stage, it was decided to import core entities like Passenger, Journey, Route, Stop, Vehicle and Feedback as graph nodes. All nodes received an ID to make sure each node was unique, to match relations to each other easily.

In phase 2, nodes were connected in pairs by matching with identifiers. Relationships like MADE, STARTED\_AT, ENDED\_AT, USES\_ROUTE, USES\_VEHICLE, OPERATES\_ON, SERVES, CONNECTED\_TO, SUBMITTED, ABOUT\_JOURNEY, ABOUT\_STOP, or ABOUT\_ROUTE have been established to represent the interactions within the transportation network.

Ingestion was done via batch operations with Python scripts so that the data would be loaded efficiently and the data consistency and integrity were ensured.

**Table 5.1 Dataset Summary**

| File                 | Purpose                  |
|----------------------|--------------------------|
| passengers.csv       | Passenger information    |
| journeys.csv         | Journey records          |
| routes.csv           | Route information        |
| stops.csv            | Stop information         |
| vehicles.csv         | Vehicle information      |
| feedback.csv         | Passenger feedback       |
| stop_routes.csv      | Route-stop mapping       |
| stop_connections.csv | Stop interchange mapping |
| journey_segments.csv | Journey segment details  |

## 5.2 Code Walkthrough

### Step 1: Import Required Libraries

The starting point to implement was to import the required libraries. The CSV files were read and processed using pandas, and communication with the graph database was achieved using the Neo4j Python Driver. The tqdm package was added for progress bars while inserting the data.

```
import pandas as pd
from neo4j import GraphDatabase
from tqdm import tqdm
```

### Step 2: Dataset Analysis

Each dataset was first analyzed for the existence of column names, data types, and possible entity relationships before the ingestion. Analysis of passengers, journeys and routes, stops, vehicles,

feedback records, stop-route mappings, stop connections and journey segments. This initial step helped to ensure that the graph model matched the data.

### Step 3: Establishing a Neo4j Connection

The Python application created a connection to Neo4j using the Bolt protocol. Since the connection was made before any database operations were performed, it was checked to ensure the graph database was available.

```
URI = "neo4j+s://be0a3f61.databases.neo4j.io"

USERNAME = "be0a3f61"
PASSWORD = "wcEdfl7sSqjEjtNsdGiLIiUwZqZ7m2Q30wVz47a3N24"

driver = GraphDatabase.driver(
    URI,
    auth=(USERNAME, PASSWORD)
)

driver.verify_connectivity()

print("Connected Successfully")

Connected Successfully
```

### Step 4: Database Initialization and Constraint Creation

Existing contents of the database were deleted to provide a clean environment for ingestion. All of the primary identifiers `passenger_id`, `journey_id`, `route_id`, `stop_id`, `vehicle_id`, and `feedback_id` were then given uniqueness constraints. These restrictions prevented duplicate nodes from being created and ensured the integrity of the data.

```
constraints = [

    """
    CREATE CONSTRAINT passenger_id_unique
    IF NOT EXISTS
    FOR (p:Passenger)
    REQUIRE p.passenger_id IS UNIQUE
    """,

    """
```

### Step 5: Loading CSV Files

The six main things that exist as datasets were put into dataframes in Pandas:

- passengers.csv
- journeys.csv
- routes.csv
- stops.csv

- vehicles.csv
- feedback.csv

There were also some other relationship datasets loaded:

- journey\_segments.csv
- stop\_routes.csv
- stop\_connections.csv

## Step 6: Node Creation

To prevent duplication in the datasets, the Neo4j MERGE operation was used to create graph nodes for each dataset. The following node labels were created:

- Passenger
- Journey
- Route
- Stop
- Vehicle
- Feedback

```
query = """
MERGE (p:Passenger {
    passenger_id:$passenger_id
})
SET p += $props
"""

with driver.session() as session:

    for _, row in tqdm(
        passengers.iterrows(),
        total=len(passengers)
    ):

        props = row.to_dict()

        props["preferred_mode"] = \
            str(props["preferred_mode"]).split("|")

        session.run(
            query,
            passenger_id=row["passenger_id"],
            props=props
        )

print("Passengers Imported")
```

100%|██████████| 500/500 [01:58<00:00, 4.23it/s]Passengers Imported

Each CSV record's properties were directly added to the corresponding node.

## Step 7: Relationship Construction

Once all nodes were added, links to entities were added. For the implementation, there were twelve types of relationships:

- MADE
- STARTED\_AT
- ENDED\_AT
- USES\_ROUTE
- USES\_VEHICLE
- OPERATES\_ON
- SERVES
- CONNECTED\_TO
- SUBMITTED
- ABOUT\_JOURNEY
- ABOUT\_STOP
- ABOUT\_ROUTE

SERVES relationships were created from the stop\_routes dataset, and STOP\_CONNECTED\_TO were created for the stop\_connections dataset, which included the property connected\_to. Merchant CONNECTED\_TO relationships included more properties such as the distance and transfer mode. Journey segments had been identified as being in either of the three relationships to journeys, vehicles, or routes (UES\_ROUTE, USES\_VEHICLE, or OPERATES\_ON).

In addition, the relationships of STARTED\_AT and ENDED\_AT relationships were created by using origin\_stop\_id and destination\_stop\_id pairs of the Journey nodes and pairing them with the Stop nodes in the Trip node, creating the full path or journey of each Trip.

## Step 8: Data Validation

After ingestion, some Cypher queries are run to ensure that there are a correct number of nodes, number of relationships, and sample number of records. This verification process verified that it was the correct transportation network defined in SUMAP and all relationships were correctly connected in the graph.

Overall, the ingestion workflow implemented was successfully able to convert the nine CSV files into a connected property graph, which serves as a solid groundwork for graph analytics on the resulting data and exploratory trips through the transportation network.



### 5.3 Sample Data Verification

Following ingestion, a number of verification queries were run to validate successful creation of nodes and relationships in the Neo4j database.

#### Verification 1: Node Counts

To find out how many nodes are being imported to the graph database, the following query was used:

```
MATCH (n) RETURN labels(n)[0] AS Label, count(*) AS Total ORDER BY Total DESC;
```

All of the six node labels were successfully created, as verified by the results. There are 3000 Journey nodes, 500 Passenger nodes, 300 Feedback nodes and other nodes (Route, Stop, Vehicle).

**Table 5.1: Sample Node Counts**

|   | Label       | Total |
|---|-------------|-------|
| 1 | "Journey"   | 3000  |
| 2 | "Passenger" | 500   |
| 3 | "Feedback"  | 300   |
| 4 | "Vehicle"   | 250   |
| 5 | "Stop"      | 120   |
| 6 | "Route"     | 21    |

#### Verification 2: Relationship Counts

The below query was run to ensure the graph connectivity:

```
MATCH ()-[r]->() RETURN type(r), count(*) ORDER BY type(r);
```

The results showed that all twelve of the relations of the studied relationship types were indeed formed.

**Table 5.2: Relationship Verification**

|    | type(r)         | count(*) |
|----|-----------------|----------|
| 1  | "ABOUT_JOURNEY" | 214      |
| 2  | "ABOUT_ROUTE"   | 161      |
| 3  | "ABOUT_STOP"    | 209      |
| 4  | "CONNECTED_TO"  | 38       |
| 5  | "ENDED_AT"      | 3000     |
| 6  | "MADE"          | 3000     |
| 7  | "OPERATES_ON"   | 177      |
| 8  | "SERVES"        | 229      |
| 9  | "STARTED_AT"    | 3000     |
| 10 | "SUBMITTED"     | 300      |
| 11 | "USES_ROUTE"    | 3054     |
| 12 | "USES_VEHICLE"  | 3073     |

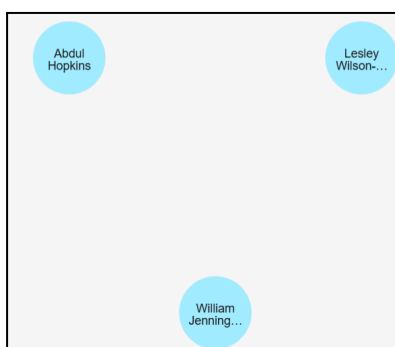
Inconsistencies in the source data were cited as a cause of the relatively small number of MADE and SUBMITTED relationships and the lack of implementation-related problems. The relatively small number of MADE and SUBMITTED relationships was reported due to inconsistencies in identifiers in the source data set and not implementation issues. Logic handling the relationship created was tested and proved to be correct for all matching records.

### Verification 3: Sample Record Retrieval

Samples of queries for retrieving nodes were run to ensure proper storage of properties.

Example:

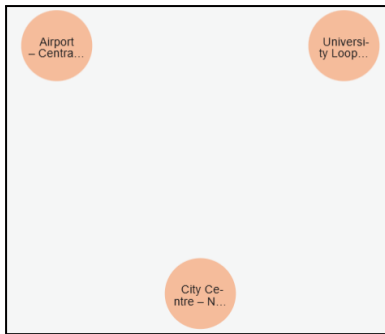
```
MATCH (p:Passenger) RETURN p LIMIT 3;
```



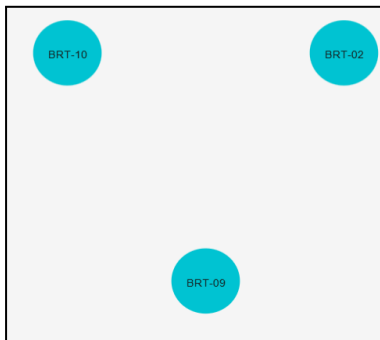
The same queries were implemented for Route and Vehicle nodes:

Example

```
MATCH (r:Route) RETURN r LIMIT 3;
```



```
MATCH (v:Vehicle) RETURN v LIMIT 3;
```



The outputs showed that node properties, including identifiers, attributes of operation, demographic properties, and transportation metadata, were captured accurately when being ingested.

#### Verification 4: Schema Visualization

The final structure of the graph was validated using Neo4j's schema visualization capabilities. The visualization featured six node labels and twelve relationships, including relationships of StartDate and EndDate such as startedAt and endedAt, which model the origin and destination of a journey.

The final graph model successfully captured the passengers, journeys, routes, stops, vehicles and feedback interactions in the SUMAP transportation ecosystem.

Finally, the ingestion pipeline was verified to successfully consume the CSV files and build a connected graph in the Neo4j database that matched both the project specs as well as the desired graph schema.

## 6.0 Analytical Queries

### 6.1 Analytical Queries (AR-01 to AR-10 Code and Explanations)

Looking about the 10 questions of query, it achieves fairly some business problems. AR01 is that what are the Top 10 busiest stops by total passenger boarding across all transport modes, broken down by zone? For this one, the code from Figure 6.1 reaches the goal. From the result, it can be seen about top 10 busiest stops.

```

# Execute query
query = """
MATCH (j:Journey)-[sa:STARTED_AT]->(s:Stop)
RETURN s.zone AS zone,
        s.stop_name AS stop,
        COUNT(j) AS total_boardings
ORDER BY total_boardings DESC
LIMIT 10
"""

# Run query and get results
results = graph.run(query).data()

# Convert to DataFrame
df = pd.DataFrame(results)
print(df)

```

| zone     | stop                  | total_boardings |
|----------|-----------------------|-----------------|
| 0 Zone A | Central Interchange   | 113             |
| 1 Zone F | Kepong LRT Station    | 72              |
| 2 Zone A | Medical Hub Station   | 70              |
| 3 Zone A | Heritage District Hub | 65              |
| 4 Zone F | Kepong Bus Stop       | 63              |
| 5 Zone F | Jenjarom LRT Station  | 61              |
| 6 Zone B | West Market Station   | 56              |
| 7 Zone A | Hotel District Hub    | 54              |
| 8 Zone D | Ledang BRT Terminal   | 52              |
| 9 Zone A | Kluang LRT Station    | 50              |

Figure 6.1 The Code And Result of AR-01

AR02 is that for each day of the week (Monday to Sunday), which two-hour time window sees the highest number of journey departures on BRT routes? For this one, the code from Figure 6.2 reaches the goal. From the result, it can be seen about 134 departures on BRT routes in 18:00-20:00 time window.

```

# Execute query
query = """
MATCH (j:Journey)-[i:USES_ROUTE]->(r:Route)
WHERE r.mode = 'BRT' OR r.route_id STARTS WITH 'BRT'
WITH j,
    substring(j.journey_date, 0, 10) AS date_part,
    // Extract hour (positions 11-12 in format "yyyy-MM-ddTHH:mm:ss")
    substring(j.journey_date, 11, 2) AS hour
WITH date_part, hour,
    COUNT(*) AS departures_per_hour
// Group into two-hour windows
WITH
    date_part,
    CASE
        WHEN toInteger(hour) >= 0 AND toInteger(hour) < 2 THEN '00:00-02:00'
        WHEN toInteger(hour) >= 2 AND toInteger(hour) < 4 THEN '02:00-04:00'
        WHEN toInteger(hour) >= 4 AND toInteger(hour) < 6 THEN '04:00-06:00'
        WHEN toInteger(hour) >= 6 AND toInteger(hour) < 8 THEN '06:00-08:00'
        WHEN toInteger(hour) >= 8 AND toInteger(hour) < 10 THEN '08:00-10:00'
        WHEN toInteger(hour) >= 10 AND toInteger(hour) < 12 THEN '10:00-12:00'
        WHEN toInteger(hour) >= 12 AND toInteger(hour) < 14 THEN '12:00-14:00'
        WHEN toInteger(hour) >= 14 AND toInteger(hour) < 16 THEN '14:00-16:00'
        WHEN toInteger(hour) >= 16 AND toInteger(hour) < 18 THEN '16:00-18:00'
        WHEN toInteger(hour) >= 18 AND toInteger(hour) < 20 THEN '18:00-20:00'
        WHEN toInteger(hour) >= 20 AND toInteger(hour) < 22 THEN '20:00-22:00'
        WHEN toInteger(hour) >= 22 AND toInteger(hour) <= 23 THEN '22:00-24:00'
        ELSE 'Unknown'
    END AS time_window,
    SUM(departures_per_hour) AS total_departures
WITH date_part, time_window, total_departures
WITH time_window,
    SUM(total_departures) AS total_departures
RETURN time_window,
        total_departures
ORDER BY total_departures DESC
LIMIT 1
"""

# Run query and get results
results = graph.run(query).data()

# Convert to DataFrame
df = pd.DataFrame(results)
print(df)

```

| time_window   | total_departures |
|---------------|------------------|
| 0 18:00-20:00 | 134              |

Figure 6.2 The Code And Result of AR-02

AR03 is that which routes have the highest average delay in minutes, and what percentage of their journey are delayed by more than 15 minutes. For this one, the code from Figure 6.3 reaches the goal. From the result, it can be seen about 'University Loop – Zone C' have highest average delay in minutes and over delayed 15 minutes journey, 19.81% occupies on total journey.

```

# Execute query
query = """
MATCH (j:Journey)-[i:USES_ROUTE]->(r:Route)
WHERE j.delay_minutes IS NOT NULL
WITH r.route_name AS Route_Name,
    avg(j.delay_minutes) AS avg_delay,
    count(*) AS total_journeys,
    count(CASE WHEN j.delay_minutes > 15 THEN 1 END) AS delayed_over_15
WHERE total_journeys >= 10
RETURN Route_Name,
        round(avg_delay, 2) AS average_delay_minutes,
        round(100.0 * delayed_over_15 / total_journeys, 2) AS pct_delayed_over_15
ORDER BY average_delay_minutes DESC
LIMIT 1
"""

# Run query and get results
results = graph.run(query).data()

# Convert to DataFrame
df = pd.DataFrame(results)
print(df)

```

| Route_Name                 | average_delay_minutes | pct_delayed_over_15 |
|----------------------------|-----------------------|---------------------|
| 0 University Loop – Zone C | 14.96                 | 19.81               |

Figure 6.3 The Code And Result of AR-03

AR04 is what is the revenue contribution (total fare collected) per commuter type (Daily, Student, Tourist, etc.) per quarter in 2023. From Figure 6.4, it shows that the total revenue of public transport every quarter of 2023

```
# Execute query
query = """
MATCH (p:Passenger)-[:MADE]->(j:Journey)
WITH p, j
    j.total_fare_myf AS fare
    j.transport_mode AS mode
WHERE j.status <> "Cancelled"

WITH p, fare, mode
CASE WHEN mode = "Multimodal"
    THEN "Multimodal"
    ELSE "Single-mode"
END AS transport_mode

RETURN
    transport_mode,
    count(DISTINCT CASE WHEN transport_mode = "Multimodal" THEN p END) AS multimodalPassenger,
    round(avg(toFloat(fare)), 2) AS avgfare,
    round(sum(toFloat(fare)), 2) AS totalfare
"""

# Run query and get results
results = graph.run(query).data()

# Convert to DataFrame
df = pd.DataFrame(results)
print(df)
```

|    | commuterType | quarter | totalRevenue |
|----|--------------|---------|--------------|
| 0  | Daily        | Q1      | 337.27       |
| 1  | Student      | Q1      | 109.05       |
| 2  | Occasional   | Q1      | 91.88        |
| 3  | Senior       | Q1      | 54.77        |
| 4  | Tourist      | Q1      | 35.51        |
| 5  | Daily        | Q2      | 307.05       |
| 6  | Student      | Q2      | 115.86       |
| 7  | Occasional   | Q2      | 104.59       |
| 8  | Senior       | Q2      | 59.90        |
| 9  | Tourist      | Q2      | 54.51        |
| 10 | Daily        | Q3      | 293.42       |
| 11 | Student      | Q3      | 116.97       |
| 12 | Occasional   | Q3      | 90.81        |
| 13 | Senior       | Q3      | 56.46        |
| 14 | Tourist      | Q3      | 49.89        |
| 15 | Daily        | Q4      | 270.00       |
| 16 | Student      | Q4      | 118.19       |
| 17 | Occasional   | Q4      | 116.87       |
| 18 | Tourist      | Q4      | 69.73        |
| 19 | Senior       | Q4      | 44.17        |

Figure 6.4 The Code And Result of AR-04

AR05 is how many unique passengers completed at least one multimodal journey(2+transport modes) in a single trip, and what is their average total fare vs single -mode journeys? From Figure 6.5, it is seen that 388 passengers completed on multiple mode journey with 3.67 averagefare and 2796.29 total fare.

```
# Execute query
query = """
MATCH (p:Passenger)-[:MADE]->(j:Journey)
WITH p, j
    j.total_fare_myf AS fare,
    j.transport_mode AS mode
WHERE j.status <> "Cancelled"

WITH p, fare, mode
CASE WHEN mode = "Multimodal"
    THEN "Multimodal"
    ELSE "Single-mode"
END AS transport_mode

RETURN
    transport_mode,
    count(DISTINCT CASE WHEN transport_mode = "Multimodal" THEN p END) AS multimodalPassenger,
    round(avg(toFloat(fare)), 2) AS avgfare,
    round(sum(toFloat(fare)), 2) AS totalfare
"""

# Run query and get results
results = graph.run(query).data()

# Convert to DataFrame
df = pd.DataFrame(results)
print(df)
```

|   | transport_mode | multimodalPassengers | avgfare | totalfare |
|---|----------------|----------------------|---------|-----------|
| 0 | Single-mode    | 0                    | 1.59    | 3004.57   |
| 1 | Multimodal     | 388                  | 3.67    | 2796.29   |

Figure 6.5 The Code And Result of AR-05

AR06 is that calculate the total and average CO2 equivalent(in kg) generate per month per transport mode. Which mode has the lowest carbon intensity per km? From Figure 6.6, it is seen that LRT shares the lowest carbon intensity every 1KM, and it also displays the result of total average CO2 and total ones.

```
# Execute query
query = """
MATCH (j:Journey)-[:USE5_ROUTE]->(r:Route)
WITH
    datetime(j.journey_date) AS dt,
    j.transport_mode AS mode,
    j.carbon_g AS co2_pj,
    r.distance_km AS km
WHERE j.status <> "Cancelled"

WITH
    dt.year AS year,
    dt.month AS month,
    mode,
    (co2_pj / 1000) AS co2_kg

RETURN
    year,
    month,
    mode AS transportMode,
    round(sum(co2_kg), 2) AS totalCO2_kg,
    round(avg(co2_kg), 2) AS avgCO2_kg
ORDER BY year, month, transportMode;
"""

# Run query and get results
results = graph.run(query).data()

# Convert to DataFrame
df = pd.DataFrame(results)
print(df)
```

|     | year | month | transportMode | totalCO2_kg | avgCO2_kg |
|-----|------|-------|---------------|-------------|-----------|
| 0   | 2022 | 1     | BRT           | 17.83       | 1.11      |
| 1   | 2022 | 1     | LRT           | 10.25       | 0.51      |
| 2   | 2022 | 1     | Multimodal    | 28.65       | 0.99      |
| 3   | 2022 | 1     | Shuttle       | 13.78       | 1.53      |
| 4   | 2022 | 2     | BRT           | 19.72       | 0.79      |
| ... | ...  | ...   | ...           | ...         | ...       |
| 139 | 2024 | 11    | Shuttle       | 10.91       | 1.56      |
| 140 | 2024 | 12    | BRT           | 21.99       | 1.16      |
| 141 | 2024 | 12    | LRT           | 9.56        | 0.56      |
| 142 | 2024 | 12    | Multimodal    | 14.66       | 0.70      |
| 143 | 2024 | 12    | Shuttle       | 14.43       | 1.60      |

[144 rows x 5 columns]

```

# Execute query
query = """
MATCH (j:Journey)-[:USES_ROUTE]->(r:Route)
WITH
    j.transport_mode AS mode,
    j.carbon_g AS co2_pj,
    r.distance_km AS km
WHERE j.status <> "Cancelled"

WITH
    mode,
    sum(co2_pj) / sum(km) AS intensity_pkm

RETURN
    mode AS transportMode,
    round(intensity_pkm, 2) AS co2_per_km
ORDER BY co2_per_km ASC
LIMIT 1
"""

# Run query and get results
results = graph.run(query).data()

# Convert to DataFrame
df = pd.DataFrame(results)
print(df)

```

|   | transportMode | co2_per_km |
|---|---------------|------------|
| 0 | LRT           | 18.24      |

Figure 6.6 The Code And Result of AR-06

AR07 is which route/stop combination has the most unresolved High or Critical feedback in the last 6 months? List the top 5 along with common tags? From Figure 6.7, it is seen that 'Rawang Micro-Hub', 'Yong Peng LRT Station', 'Serdang Bus Stop', 'Stadium Interchange' and 'West Market Station' have most unresolved feedback, and it can the the content complaining.

```

# Execute query
query = """
MATCH (f:Feedback)-[:ABOUT_ROUTE]->(r:Route)
MATCH (f)-[:ABOUT_STOP]->(s:Stop)
WITH
    f, r, s,
    datetime(f.submitted_at) AS dt
WHERE f.is_resolved = false
    AND f.severity IN ["High", "Critical"]
    AND dt >= datetime("2024-12-30T16:32:08") - duration("P6M")

WITH
    r.route_id AS route,
    s.stop_name AS stop,
    collect(f.tags) AS allTags,
    count(f) AS unresolvedCount

RETURN
    route,
    stop,
    unresolvedCount,
    allTags
ORDER BY unresolvedCount DESC
LIMIT 5;
"""

# Run query and get results
results = graph.run(query).data()

# Convert to DataFrame
df = pd.DataFrame(results)
print(df)

```

|   | route  | stop                  | unresolvedCount | allTags                              |
|---|--------|-----------------------|-----------------|--------------------------------------|
| 0 | BRT-02 | Rawang Micro-Hub      | 1               | [[ticketing, app-bug, peak-hour]]    |
| 1 | BRT-04 | Yong Peng LRT Station | 1               | [[praise, bike-damage]]              |
| 2 | BRT-05 | Serdang Bus Stop      | 1               | [[first-mile, last-mile, peak-hour]] |
| 3 | BRT-05 | Stadium Interchange   | 1               | [[overcrowded, peak-hour]]           |
| 4 | BRT-02 | West Market Station   | 1               | [[peak-hour, delay]]                 |

Figure 6.7 The Code And Result of AR-07

AR08 is for Gold and Platinum tier passengers, what is the average number of journeys per month, preferred transport mode, and average rating given. From Figure 6.8, it can be seen that golden passengers share 1 average journey ,multimodal of preferred transport mode, 1.67 of average rating . In other side, platinum ones share 1 average journey, multimodal of preferred transport mode, 1.97 average rating.

```

query =
MATCH (p:Passenger)-[:MADE]->(j:Journey)
WHERE p.loyalty_tier IN ['Gold', 'Platinum']
WITH p
LEFT JOIN j.journey_date j AS year_month,
j.transport_mode AS mode,
CASE
WHEN j.rating IS NULL OR tostring(j.rating) = "NaN" THEN 0.0
ELSE j.rating
END AS rating
// Per passenger per month
WITH p, year_month, mode,
COUNT(*) AS monthly_journeys,
AVG(rating) AS monthly_rating
// Per passenger
WITH p,
AVG(monthly_journeys) AS avg_monthly,
AVG(monthly_rating) AS avg_rating,
COLLECT(DISTINCT mode) AS modes_used
// Per passenger - Find preferred mode
WITH p,
avg_monthly,
avg_rating,
[ m IN modes_used WHERE m IS NOT NULL AND m <= "" ][0] AS preferred_mode
// Group by loyalty_tier
WITH p.loyalty_tier AS loyalty_tier,
ROUND(AVG(avg_monthly), 2) AS avg_journeys_per_month,
COLLECT(preferred_mode) AS all_modes,
ROUND(AVG(avg_rating), 2) AS avg_rating_given
WITH loyalty_tier,
avg_journeys_per_month,
avg_rating_given,
all_modes,
// Find most common preferred mode
REDUCE(best = (mode: "", count: 0), m IN all_modes |
CASE
WHEN m IS NOT NULL AND m <= "" AND SIZE([x IN all_modes WHERE x = m]) > best.count
THEN (mode: m, count: SIZE([x IN all_modes WHERE x = m]))
ELSE best
END
) AS top_mode
RETURN loyalty_tier,
avg_journeys_per_month,
top_mode AS preferred_transport_mode,
avg_rating_given
ORDER BY loyalty_tier
---
```

|   | loyalty_tier | avg_journeys_per_month | preferred_transport_mode |
|---|--------------|------------------------|--------------------------|
| 0 | Gold         | 1.00                   | Multimodal               |
| 1 | Platinum     | 1.02                   | Multimodal               |

```

# Run query and get results
results = graph.run(query).data()
# Convert to DataFrame
df = pd.DataFrame(results)
print(df)

```

|   | avg_rating_given |
|---|------------------|
| 0 | 1.67             |
| 1 | 1.97             |

Figure 6.8 The Code And Result of AR-08

AR09 is that using the interchange connection data in the database, find all stops reachable from Central Interchange (STOP-001) within 2 transfer hops. For each reachable stop, return its name, zone, and the routes available at that stop. From Figure 6.9, it can be seen that there are 51 stops can reach the goal.

```

# Execute query
query = """
MATCH (start:Stop {stop_id: "STOP-001"})
CALL {
WITH start
// 1 hop
MATCH (start)-[:SERVES]->(r:Route)-[:SERVES]->(stop2:Stop)
WHERE stop2 < start
RETURN DISTINCT stop2 AS stop, r AS route, 1 AS hops
UNION
// 2 hops
MATCH (start)-[:SERVES]->(r1:Route)-[:SERVES]->(mid:Stop)-[:SERVES]->(r2:Route)-[:SERVES]->(stop2:Stop)
WHERE mid < start
AND stop2 < start
AND stop2 < mid
RETURN DISTINCT stop2 AS stop, r2 AS route, 2 AS hops
}
WITH stop, hops, COLLECT(DISTINCT route) AS routes
RETURN
stop.stop_name AS stop_name,
stop.zone AS zone,
[ r IN routes | r.route_id ] AS route_ids,
[ r IN routes | r.route_name ] AS route_names
ORDER BY stop, stop_name
---
```

|    | stop_name                 | zone   | route_ids             | route_names                                         |
|----|---------------------------|--------|-----------------------|-----------------------------------------------------|
| 0  | Airport Terminal Hub      | Zone F | [LRT-A]               |                                                     |
| 1  | Ampang Pecah Micro-Hub    | Zone C | [BRT-11]              |                                                     |
| 2  | Bahau Bus Stop            | Zone F | [BRT-06, LRT-A]       | [Green Line - Airport to Northgate]                 |
| 3  | Bangi Micro-Hub           | Zone E | [BRT-01]              | [Medical Hub - Central Interchange]                 |
| 4  | Buloh Kasap Bus Stop      | Zone F | [LRT-A]               | [Central Interchange - Tech Park, Green Line - ...] |
| 5  | Cheras LRT Station        | Zone F | [LRT-B]               | [City Centre - Northgate Express]                   |
| 6  | Desa Tebrau BRT Terminal  | Zone D | [LRT-C]               | [Green Line - Airport to Northgate]                 |
| 7  | Desa Tebrau Bus Stop      | Zone B | [BRT-06, LRT-A]       | [Blue Line - East-West Crosstown]                   |
| 8  | East Terminal Interchange | Zone E | [LRT-B]               | [Red Line - Central Loop]                           |
| 9  | Gemas North LRT Station   | Zone F | [BRT-01]              | [Central Interchange - Tech Park, Green Line - ...] |
| 10 | Genuang BRT Terminal      | Zone F | [BRT-01]              | [Blue Line - East-West Crosstown]                   |
| 11 | Gombak BRT Terminal       | Zone C | [BRT-06]              | [City Centre - Northgate Express]                   |
| 12 | Heritage District Hub     | Zone A | [LRT-C]               | [City Centre - Northgate Express]                   |
| 13 | Hotel District Hub        | Zone A | [BRT-01, LRT-C]       | [City Centre - Northgate Express]                   |
| 14 | Hulu Langat LRT Station   | Zone E | [LRT-A]               | [Central Interchange - Tech Park]                   |
| 15 | Janjarom LRT Station      | Zone F | [LRT-B, LRT-C]        | [Red Line - Central Loop]                           |
| 16 | Kepong Bus Stop           | Zone F | [LRT-C]               | [City Centre - Northgate Express, Red Line - C ...] |
| 17 | Kepong LRT Station        | Zone F | [LRT-A, LRT-B, LRT-C] | [Green Line - Airport to Northgate]                 |
| 18 | Kluang LRT Station        | Zone A | [LRT-A, LRT-B]        | [Blue Line - East-West Crosstown, Red Line - C ...] |
| 19 | Kuala Klawang Bus Stop    | Zone A | [BRT-01]              | [Red Line - Central Loop]                           |
| 20 | Lakeview Interchange      | Zone D | [LRT-A]               | [Green Line - Airport to Northgate, Blue Line ...]  |
| 21 | Larkin Bus Stop           | Zone C | [BRT-11, LRT-B]       | [Green Line - Airport to Northgate, Blue Line ...]  |
| 22 | Ledang BRT Terminal       | Zone D | [LRT-A, LRT-B]        | [City Centre - Northgate Express]                   |
| 23 | Lenggeng LRT Station      | Zone F | [LRT-A, LRT-B]        | [Green Line - Airport to Northgate]                 |
| 24 | Linggi Beach Bus Stop     | Zone F | [LRT-A]               | [Green Line - Airport to Northgate]                 |
| 25 | Linggi Bus Stop           | Zone D | [BRT-06]              | [Medical Hub - Central Interchange, Blue Line ...]  |
| 26 | Masai Bus Stop            | Zone C | [BRT-06]              | [Green Line - Airport to Northgate, Blue Line ...]  |

Figure 6.9 The Code And Result of AR-09

AR10 is that identify passengers who have more than 20% of their journeys with status 'Cancelled' or 'Incomplete' in 2024, along with their total journey count and home zone. From Figure 6.10, it can be seen that there are 239 passengers have more than 20% of their journeys with status 'Cancelled' or 'Incomplete' in 2024.

```

# Execute query
query = """
MATCH (p:Passenger)-[:MADE]->(j:Journey)
WHERE j.journey_date STARTS WITH '2024'
WITH p,
COUNT(j) AS total_journeys,
SUM(CASE WHEN j.status IN ['Cancelled', 'Incomplete'] THEN 1 ELSE 0 END) AS bad_journeys
WHERE total_journeys > 0 // Avoid division by zero
AND (bad_journeys * 1.0 / total_journeys) > 0.20
RETURN p.passenger_id AS passenger_id,
p.full_name AS full_name,
p.home_zone AS home_zone,
total_journeys,
bad_journeys AS cancelled_incomplete_count,
ROUND((bad_journeys * 100.0 / total_journeys), 2) AS bad_percentage
ORDER BY bad_percentage DESC
---
```

|     | passenger_id                          | full_name            | home_zone |
|-----|---------------------------------------|----------------------|-----------|
| 0   | fc61a281-2d53-4810-bc01-c46c54ac13ce  | Ms Hilary Walters    | Zone A    |
| 1   | 702805af-5906-41f1-b820-242631341ab0  | Joel Stephens        | Zone B    |
| 2   | 5f5c77f4-d7a9-48ea-af7c-db39283edeec  | Terry Smith-Harrison | Zone D    |
| 3   | f20ba0c9-2a86-4922-b492-7b3536c21bc3  | Lydia Bishop         | Zone D    |
| 4   | 970e287a-0fd7-40f2-8f90-8490006e0070  | Patrick Jones        | Zone F    |
| ... | ...                                   | ...                  | ...       |
| 234 | 98ab182c-3b1d-4181-9cc3-c1f59129bc75  | Miss Alice Tomlinson | Zone B    |
| 235 | b1f94a09-7408-4646-93a7-1719bdcdf8302 | Benjamin Peters      | Zone A    |
| 236 | d94296b6-7e4b-4f10-b45f-780ef5b5403b  | Dr Mathew Fowler     | Zone A    |
| 237 | 89b4db00-8a32-484a-a481-461af5b92b3c  | Terence Davis        | Zone E    |
| 238 | 29769b0f-41e0-4faf-92ca-9cedc5268f0f  | Stephanie Lloyd      | Zone A    |

|     | total_journeys | cancelled_incomplete_count | bad_percentage |
|-----|----------------|----------------------------|----------------|
| 0   | 2              | 2                          | 100.0          |
| 1   | 2              | 2                          | 100.0          |
| 2   | 2              | 2                          | 100.0          |
| 3   | 1              | 1                          | 100.0          |
| 4   | 2              | 2                          | 100.0          |
| ... | ...            | ...                        | ...            |
| 234 | 4              | 1                          | 25.0           |
| 235 | 4              | 1                          | 25.0           |
| 236 | 4              | 1                          | 25.0           |
| 237 | 4              | 1                          | 25.0           |
| 238 | 4              | 1                          | 25.0           |

```

# Run query and get results
results = graph.run(query).data()
# Convert to DataFrame
df = pd.DataFrame(results)
print(df)

```

Figure 6.10 The Code And Result of AR-10

## 6.2 Query Performance Discussion

Ensuring the effective performance of the neo4j, it is crucial to identify which queries share the expensive cost and tune the expensive ones at the same time. In neo4j, usually it can be considered as expensive query which scans multiple nodes and relationships, and aggregates more data.

Based on these rules, it can be concluded that AR08 share the most expensive cost, compared to other 8 ones. First of all, it can find that 'Passengers' node has 500 rows, 'Journeys' has 3000 rows, 'Routes' has 21 rows, 'Stops' has 120 rows, 'Vehicles' has 250 rows, and 'Feedback' has 300 rows. It suggests that the combination of 'Journeys' and 'Passengers' can consume most expensive resource. It can make an early decision that AR04, AR05, AR08 and AR10 are more expensive than others.

Next step, moving to the quantity of calculation during execution. Checking with the queries from these four questions, AR08 consumes more aggregations than others. So, it can be concluded that AR08 are the most expensive query, compared to other questions.

## 7.0 System Reflection & Evaluation

This section critically evaluates the Neo4j implementation of SUMAP. The graph model proved well suited to the connected and variable structure of Nusaville's transport data, yet no single database is ideal for every type of workload. The following subsections discuss the main trade-offs of using Neo4j for this use case (Section 7.1), how the system is expected to behave as the data volume grows (Section 7.2), and how the design would have differed if MongoDB had been chosen instead (Section 7.3).

### 7.1 NoSQL Trade-offs

Even though Neo4j fits the relationship-intensive nature of SUMAP, the implementation also exposed two practical trade-offs that should be acknowledged.

The first trade-off concerns analytical aggregation. Most of the analytical requirements are aggregation queries rather than traversal queries, and aggregation is not the area in which a graph database is strongest (Robinson, Webber, & Eifrem, 2015). Of the ten requirements, only AR-09, which finds the stops reachable from Central Interchange within two transfer hops, and parts of AR-05 and AR-07 genuinely depend on traversing relationships. The remaining requirements, including AR-01 ridership by zone, AR-02 the peak two-hour window per weekday, AR-03 average delay per route, AR-04 revenue per commuter type per quarter, AR-06 carbon output per month per mode, AR-08 loyalty engagement, and AR-10 the cancellation rate per passenger, are essentially



grouped aggregations over the 3,000 Journey nodes. Cypher can express these using count, sum, avg, and collect, but the engine is optimised for pointer-based traversal rather than for scanning and grouping large sets of nodes, so these queries gain little from the graph structure and tend to be more verbose than an equivalent SQL GROUP BY or MongoDB aggregation pipeline. In AR-04 and AR-06 the reporting period must also be derived from a date that is stored as a string property, which adds further work because the value has to be recomputed for every Journey node that is scanned.

The second trade-off concerns schema validation. The flexibility that makes Neo4j convenient for SUMAP also means that very little is enforced at the database level. The implementation defines only six uniqueness constraints, one on each identifier property, together with the supporting indexes on those identifiers. Beyond uniqueness, the database does not enforce property types, value ranges, or the enumerations given in the data dictionary, such as the rule that Journey.status must be Completed, Cancelled, or Incomplete, that rating must fall between 1 and 5, or that the pipe-delimited fields preferred\_mode, facilities, and tags are always split into clean lists. All of this validation is handled inside the Python ingestion script, where the pipe-separated fields are split manually before loading. A relational database, by contrast, would reject malformed rows at insertion time through column types and CHECK constraints. This limitation is visible in the project's own verification results: fewer MADE and SUBMITTED relationships were created than the number of Journey and Feedback records, because some journey and feedback rows referenced passenger identifiers that did not match any Passenger node, as noted in Section 5.3. A graph database does not enforce this kind of referential integrity, so the unmatched references simply produced no relationship instead of raising an error. This behaviour is acceptable for an analytics layer built from controlled CSV exports, but it places the responsibility for data quality on the ingestion code rather than on the database itself.

## 7.2 Scalability Considerations

The current database is small, containing 500 passengers, 3,000 journeys, 21 routes, 120 stops, 250 vehicles, and 300 feedback records, together with 229 SERVES relationships, 42 CONNECTED\_TO interchange relationships, and the journey-to-route, journey-to-vehicle, and vehicle-to-route relationships derived from the 4,365 journey segments. If the dataset were to grow tenfold, to roughly 30,000 journeys and tens of thousands of additional relationships, the system would behave unevenly depending on the type of query. Queries that are anchored on an indexed node and explore only a local neighbourhood would continue to perform well. Because every node label has a uniqueness constraint with a backing index, looking up a passenger or a stop by its identifier remains effectively constant time regardless of database size, and short traversals that start from such a node, such as the two-hop interchange search in AR-09 or the per-passenger journey counts in AR-08, visit only the immediate surroundings of that node and therefore depend on local connectivity rather than on the

total size of the graph. These are the queries that benefit most from the graph model and that scale most gracefully.

The aggregation queries identified in Section 7.1 are the ones that would degrade. AR-01, AR-03, AR-04, AR-06, and AR-10 must read every Journey node and group the results, so their cost grows roughly in proportion to the number of journeys; at ten times the data they would take on the order of ten times as long and would depend on the whole working set fitting into the page cache. The free AuraDB and Community tiers scale vertically, by using a single larger instance, rather than horizontally, and they provide no native sharding to spread this work across machines. These tiers also limit the number of nodes and relationships, so a tenfold increase could exceed the free allowance and require a paid instance with more memory.

Several measures would keep performance acceptable at a larger scale. First, the ingestion process should rely on bulk operations: loading the nodes and relationships with UNWIND, apoc.periodic.iterate, or LOAD CSV, rather than issuing a separate write for each record, keeps the number of round-trips to the database low and scales far better towards 30,000 journeys and beyond. Second, additional indexes on the properties that the aggregation queries filter and group on, such as journey\_date, status, transport\_mode, and feedback severity, would reduce the amount of scanning required. Third, frequently requested metrics could be pre-computed; maintaining denormalised counters, which the dataset already includes as Passenger.total\_journeys and Stop.avg\_daily\_boardings, or periodically materialising monthly carbon and ridership summaries would turn expensive full scans into cheap look-ups. With these changes the graph would continue to serve the connectivity workloads well while keeping the aggregation workloads within budget.

### **7.3 Comparison (MongoDB vs. Neo4j)**

If MongoDB had been chosen instead of Neo4j, the design would have been organised around documents and embedding rather than around nodes and relationships. The most natural model would store each Journey as a single document with its one to four segments embedded as an array, so that a multimodal trip, together with all of its modes, routes, vehicles, fares, and times, would be held in one document. This matches the variable n\_segments structure directly and would turn AR-05 and any per-journey read into a single document fetch, with no USES\_ROUTE or USES\_VEHICLE traversal required. The heterogeneous vehicle telemetry and the free-text feedback would also fit comfortably in MongoDB: telemetry could be a flexible embedded sub-document that varies by mode, and a text index over the feedback description and tags would support the keyword search behind AR-07.

For the seven aggregation-style requirements, MongoDB's aggregation pipeline would arguably be the better fit. Stages such as \$match, \$group, and \$bucket, together with the date operators, map cleanly onto these questions; AR-04 revenue per commuter type per quarter and AR-06 carbon per month per mode, for example, correspond almost directly to a \$group expression, and the pipeline is heavily optimised for this kind of grouped roll-up. Cross-entity questions that Neo4j answers with pattern matching would instead be handled with references and \$lookup joins, which is workable but becomes less elegant when several entities have to be chained together.

The decisive difference lies in AR-09 and the interchange network. Expressing the idea of all stops reachable within two transfer hops is a single variable-length pattern in Neo4j, written as MATCH (a:Stop)-[:CONNECTED\_TO\*1..2]->(b:Stop). MongoDB can approximate this with \$graphLookup over the stop connections, but recursive, variable-depth traversal is exactly the kind of operation that the document model handles least naturally, and its cost rises quickly as the number of hops grows or as the question shifts towards shortest paths or network centrality. Because interchange connectivity was the main reason the team adopted a graph database, Neo4j remains the better overall choice for SUMAP. MongoDB would have simplified the aggregation queries and the multimodal embedding, but at the expense of the traversal workload that defines the platform. The comparison is therefore a genuine trade-off between document-native aggregation and graph-native connectivity (Sadalage & Fowler, 2013), and the core requirements of SUMAP fall on the connectivity side.

## 8. Conclusion

This mini project designed and implemented a unified NoSQL data layer for the Smart Urban Mobility Analytics Platform using Neo4j. The final graph consolidates the six SUMAP entities into six node labels, namely Passenger, Journey, Route, Stop, Vehicle, and Feedback, connected by twelve relationship types, and it was populated from the provided CSV files with 500 passengers, 3,000 journeys, 21 routes, 120 stops, 250 vehicles, and 300 feedback records, together with the SERVES, CONNECTED\_TO, and segment-derived relationships that link them. The key design decisions were to model the entities through relationships rather than by embedding, so that the many-to-many route-and-stop network and the optional feedback links could be traversed directly; to attach the transfer properties via\_mode, avg\_transfer\_walk\_mins, and peak\_hour\_only to the CONNECTED\_TO relationship, so that the interchange network carries its own metadata; and to enforce six uniqueness constraints with backing indexes in order to guarantee integrity and fast look-ups. All ten analytical requirements, AR-01 to AR-10, were implemented as Cypher queries against this model.

The main lesson of the project is that the value of a graph database in this use case lies in connectivity rather than in counting. Neo4j expressed the interchange and multimodal-journey requirements far more naturally than a relational or document model would have, but it offered little advantage for the many aggregation-style requirements, which a document or relational system might have served more concisely. The project also showed the practical importance of efficient, batched ingestion and of validating data before loading, since the database itself enforces little beyond uniqueness. Overall, Neo4j met the SUMAP goal of providing a single, queryable, and schema-flexible transport data layer, and the exercise clarified both the strengths and the limits of graph modelling for real-world urban mobility analytics.

## References

- Buchwald, P., & Sobczak, P. (2021). Using Neo4j database and graph model for analysis of metropolitan railway connections of Silesian Voivodeship in Poland. *TASK Quarterly*, 25(4), 419–430. <https://doi.org/10.34808/tq2021/25.4/d>
- Le Hasif, C., Araldo, A., Dumbrava, S., & Watel, D. (2022). A graph-database approach to assess the impact of demand-responsive services on public transit accessibility. In *Proceedings of the 15th ACM SIGSPATIAL International Workshop on Computational Transportation Science*. Association for Computing Machinery. <https://doi.org/10.1145/3557991.3567798>
- Park, S., & Cheng, T. (2023). Framework for constructing multimodal transport networks and routing using a graph database: A case study in London. *Transactions in GIS*, 27(5), 1391–1417. <https://doi.org/10.1111/tgis.13071>
- Angles, R., Arenas, M., Barceló, P., Hogan, A., Reutter, J. L., & Vrgoč, D. (2023). Property graphs have come of age. *The VLDB Journal*, 32(6), 1237–1275. <https://doi.org/10.1007/s00778-023-00810-9>
- Neo4j, Inc. (n.d.). Build applications with Neo4j and Python. Neo4j Documentation. Retrieved June 25, 2026, from <https://neo4j.com/docs/python-manual/current/>
- Neo4j, Inc. (n.d.). What is graph data modeling? Neo4j Documentation. Retrieved June 25, 2026, from <https://neo4j.com/docs/getting-started/data-modeling/>
- Robinson, I., Webber, J., & Eifrem, E. (2015). *Graph databases: New opportunities for connected data* (2nd ed.). O'Reilly Media.
- Sadalage, P. J., & Fowler, M. (2013). *NoSQL distilled: A brief guide to the emerging world of polyglot persistence*. Addison-Wesley Professional.

## Appendices

### Appendix A: Full Ingestion Script

#### Import Passenger Nodes

```
query = """
MERGE (p:Passenger {
  passenger_id:$passenger_id
})
SET p += $props
"""

with driver.session() as session:

    for _, row in tqdm(
        passengers.iterrows(),
        total=len(passengers)
    ):

        props = row.to_dict()

        props["preferred_mode"] = \
            str(props["preferred_mode"]).split("|")

        session.run(
            query,
            passenger_id=row["passenger_id"],
            props=props
        )

    print("Passengers Imported")

100%|██████████| 500/500 [01:58<00:00, 4.23it/s]Passengers Imported
```

#### Import Journey Node

```
query = """
MERGE (j:Journey {
  journey_id:$journey_id
})
SET j += $props
"""

with driver.session() as session:

    for _, row in tqdm(
        journeys.iterrows(),
        total=len(journeys)
    ):

        session.run(
            query,
            journey_id=row["journey_id"],
            props=row.to_dict()
        )

    print("Journeys Imported")

... 100%|██████████| 3000/3000 [11:52<00:00, 4.21it/s]Journeys Imported
```

## Import Route Node

```
query = """
MERGE (r:Route {
    route_id:$route_id
})
SET r += $props
"""

with driver.session() as session:

    for _, row in tqdm(
        routes.iterrows(),
        total=len(routes)
    ):

        session.run(
            query,
            route_id=row["route_id"],
            props=row.to_dict()
        )

print("Routes Imported")

100%|██████████| 21/21 [00:05<00:00, 4.14it/s]Routes Imported
```

## Import Stop Node

```
query = """
MERGE (s:Stop {
    stop_id:$stop_id
})
SET s += $props
"""

with driver.session() as session:

    for _, row in tqdm(
        stops.iterrows(),
        total=len(stops)
    ):

        props = row.to_dict()

        props["facilities"] = \
            str(props["facilities"]).split("|")

        session.run(
            query,
            stop_id=row["stop_id"],
            props=props
        )

print("Stops Imported")

100%|██████████| 120/120 [00:30<00:00, 3.90it/s]Stops Imported
```

## Import Vehicle Node

```
query = """
MERGE (v:Vehicle {
  vehicle_id:$vehicle_id
})
SET v += $props
"""

with driver.session() as session:

    for _, row in tqdm(
        vehicles.iterrows(),
        total=len(vehicles)
    ):

        session.run(
            query,
            vehicle_id=row["vehicle_id"],
            props=row.to_dict()
        )

print("Vehicles Imported")
```

... 100%|██████████| 250/250 [01:01<00:00, 4.06it/s]Vehicles Imported

## Import Feedback Node

```
query = """
MERGE (f:Feedback {
  feedback_id:$feedback_id
})
SET f += $props
"""

with driver.session() as session:

    for _, row in tqdm(
        feedback.iterrows(),
        total=len(feedback)
    ):

        props = row.to_dict()

        props["tags"] = \
            str(props["tags"]).split("|")

        session.run(
            query,
            feedback_id=row["feedback_id"],
            props=props
        )

print("Feedback Imported")
```

100%|██████████| 300/300 [01:12<00:00, 4.12it/s]Feedback Imported

## Appendix B: All 10 Query Results (Full Output)

AR01: What are the Top 10 busiest stops by total passenger boarding across all transport modes, broken down by zone?

Cypher:

```
1 MATCH (j:Journey)-[sa:STARTED_AT]->(s:Stop)
2 RETURN s.zone AS zone,
3         s.stop_name AS stop,
4         COUNT(j) AS total_boardings
5 ORDER BY total_boardings DESC
6 LIMIT 10
```

Result:

|    | zone     | stop                    | total_boardings |
|----|----------|-------------------------|-----------------|
| 1  | "Zone A" | "Central Interchange"   | 113             |
| 2  | "Zone F" | "Kepong LRT Station"    | 72              |
| 3  | "Zone A" | "Medical Hub Station"   | 70              |
| 4  | "Zone A" | "Heritage District Hub" | 65              |
| 5  | "Zone F" | "Kepong Bus Stop"       | 63              |
| 6  | "Zone F" | "Jenjarom LRT Station"  | 61              |
| 7  | "Zone B" | "West Market Station"   | 56              |
| 8  | "Zone A" | "Hotel District Hub"    | 54              |
| 9  | "Zone D" | "Ledang BRT Terminal"   | 52              |
| 10 | "Zone A" | "Kluang LRT Station"    | 50              |

AR02: For each day of the week (Monday to Sunday), which two-hour time window sees the highest number of journey departures on BRT routes?

Cypher:

```
1 MATCH (j:Journey)-[:USES_ROUTE]->(r:Route)
2 WHERE r.mode = 'BRT' OR r.route_id STARTS WITH 'BRT'
3 WITH j,
4     substring(j.journey_date, 0, 10) AS date_part,
5     // Extract hour (positions 11-12 in format "yyyy-MM-ddTHH:mm:ss")
6     substring(j.journey_date, 11, 2) AS hour
7 WITH date_part, hour,
8     COUNT(*) AS departures_per_hour
9 // Group into two-hour windows
10 WITH
11     date_part,
12     CASE
13         WHEN toInteger(hour) >= 0 AND toInteger(hour) < 2 THEN '00:00-02:00'
14         WHEN toInteger(hour) >= 2 AND toInteger(hour) < 4 THEN '02:00-04:00'
15         WHEN toInteger(hour) >= 4 AND toInteger(hour) < 6 THEN '04:00-06:00'
16         WHEN toInteger(hour) >= 6 AND toInteger(hour) < 8 THEN '06:00-08:00'
17         WHEN toInteger(hour) >= 8 AND toInteger(hour) < 10 THEN '08:00-10:00'
18         WHEN toInteger(hour) >= 10 AND toInteger(hour) < 12 THEN '10:00-12:00'
19         WHEN toInteger(hour) >= 12 AND toInteger(hour) < 14 THEN '12:00-14:00'
20         WHEN toInteger(hour) >= 14 AND toInteger(hour) < 16 THEN '14:00-16:00'
21         WHEN toInteger(hour) >= 16 AND toInteger(hour) < 18 THEN '16:00-18:00'
```



```

23     WHEN toInteger(hour) >= 20 AND toInteger(hour) < 22 THEN '20:00-22:00'
24     WHEN toInteger(hour) >= 22 AND toInteger(hour) <= 23 THEN '22:00-24:00'
25     ELSE 'Unknown'
26   END AS time_window,
27   SUM(departures_per_hour) AS total_departures
28 WITH date_part, time_window, total_departures
29 WITH time_window,
30   SUM(total_departures) AS total_departures
31 RETURN time_window,
32        total_departures
33 ORDER BY total_departures DESC
34 LIMIT 1

```

Result:

| time_window   | total_departures |
|---------------|------------------|
| "18:00-20:00" | 134              |

AR03: Which routes have the highest average delay in minutes, and what percentage of their journey are delayed by more than 15 minutes?

Cypher:

```

1 MATCH (j:Journey)-[:USES_ROUTE]->(r:Route)
2 WHERE j.delay_minutes IS NOT NULL
3 WITH r.route_name AS Route_Name,
4      avg(j.delay_minutes) AS avg_delay,
5      count(*) AS total_journeys,
6      count(CASE WHEN j.delay_minutes > 15 THEN 1 END) AS delayed_over_15
7 WHERE total_journeys >= 10
8 RETURN Route_Name,
9        round(avg_delay, 2) AS average_delay_minutes,
10       round(100.0 * delayed_over_15 / total_journeys, 2) AS pct_delayed_over_15
11 ORDER BY average_delay_minutes DESC
12 LIMIT 1

```

Result:

| Route_Name                 | average_delay_minutes | pct_delayed_over_15 |
|----------------------------|-----------------------|---------------------|
| "University Loop - Zone C" | 14.96                 | 19.81               |

AR04: What is the revenue contribution (total fare collected) per commuter type (Daily, Student, Tourist, etc.) per quarter in 2023?

Cypher:

```

1 MATCH (p:Passenger)-[:MADE]->(j:Journey)
2 WITH p.commuter_type AS commuterType,
3      datetime(j.journey_date) AS journey_DT,
4      j.total_fare_myr AS fare
5 WHERE journey_DT.year = 2023
6
7 RETURN
8   commuterType,
9   'Q' + toString(toInteger(ceil(journey_DT.month / 3.0))) AS quarter,
10  round(SUM(fare), 2) AS totalRevenue
11 ORDER BY quarter, totalRevenue DESC;

```

Result:

|    | commuterType | quarter | totalRevenue |
|----|--------------|---------|--------------|
| 1  | "Daily"      | "Q1"    | 337.27       |
| 2  | "Student"    | "Q1"    | 109.05       |
| 3  | "Occasional" | "Q1"    | 91.88        |
| 4  | "Senior"     | "Q1"    | 54.77        |
| 5  | "Tourist"    | "Q1"    | 35.51        |
| 6  | "Daily"      | "Q2"    | 307.05       |
| 7  | "Student"    | "Q2"    | 115.86       |
| 8  | "Occasional" | "Q2"    | 104.59       |
| 9  | "Senior"     | "Q2"    | 59.9         |
| 10 | "Tourist"    | "Q2"    | 54.51        |

|    | commuterType | quarter                                                                                 | totalRevenue |
|----|--------------|-----------------------------------------------------------------------------------------|--------------|
| 11 | "Daily"      | "Q3"                                                                                    | 293.42       |
| 12 | "Student"    |  "Q3" | 116.97       |
| 13 | "Occasional" | "Q3"                                                                                    | 90.81        |
| 14 | "Senior"     | "Q3"                                                                                    | 56.46        |
| 15 | "Tourist"    | "Q3"                                                                                    | 49.89        |
| 16 | "Daily"      | "Q4"                                                                                    | 270.0        |
| 17 | "Student"    | "Q4"                                                                                    | 118.19       |
| 18 | "Occasional" | "Q4"                                                                                    | 116.87       |
| 19 | "Tourist"    | "Q4"                                                                                    | 69.73        |
| 20 | "Senior"     | "Q4"                                                                                    | 44.17        |

AR05 : How many unique passengers completed at least one multimodal journey(2+transport modes) in a single trip, and what is their average total fare vs single-mode journeys?

Cypher:

```

1 MATCH (p:Passenger)-[:MADE]->(j:Journey)
2 WITH p,
3     j.total_fare_myrr AS fare,
4     j.transport_mode AS mode
5 WHERE j.status <> "Cancelled"
6
7 WITH p, fare, mode,
8     CASE WHEN mode = "Multimodal"
9         THEN "Multimodal"
10        ELSE "Single-mode"
11        END AS transport_mode
12
13 RETURN
14     transport_mode,
15     count(DISTINCT CASE WHEN transport_mode = "Multimodal" THEN p END) AS multimodalPassengers,
16     round(avg(toFloat(fare)),2) AS avgFare,
17     round(sum(toFloat(fare)),2) AS totalFare

```

Result:

| transport_mode | multimodalPassengers | avgFare | totalFare |
|----------------|----------------------|---------|-----------|
| "Single-mode"  | 0                    | 1.59    | 3004.57   |
| "Multimodal"   | 388                  | 3.67    | 2796.29   |

Ar06: Calculate the total and average CO2 equivalent(in kg) generate per month per transport mode.Which mode has the lowest carbon intensity per km?

Cypher:

For total CO2 per month and per transport mode

```

1 MATCH (j:Journey)-[:USES_ROUTE]->(r:Route)
2 WITH
3     datetime(j.journey_date) AS dt,
4     j.transport_mode AS mode,
5     j.carbon_g AS co2_pJ,
6     r.distance_km AS km
7 WHERE j.status <> "Cancelled"
8
9 WITH
10     dt.year AS year,
11     dt.month AS month,
12     mode,
13     (co2_pJ / 1000) AS co2_kg
14
15 RETURN
16     year,
17     month,
18     mode AS transportMode,
19     round(sum(co2_kg), 2) AS totalCO2_kg,
20     round(avg(co2_kg), 2) AS avgCO2_kg
21 ORDER BY year, month, transportMode;

```

For lowest carbon intensity per km

```

1 MATCH (j:Journey)-[:USES_ROUTE]->(r:Route)
2 WITH
3     j.transport_mode AS mode,
4     j.carbon_g AS co2_pJ,
5     r.distance_km AS km
6 WHERE j.status <> "Cancelled"
7
8 WITH
9     mode,
10    sum(co2_pJ) / sum(km) AS intensity_pkm
11
12 RETURN
13     mode AS transportMode,
14     round(intensity_pkm, 2) AS co2_per_km
15 ORDER BY co2_per_km ASC
16 LIMIT 1

```

Result:

For part result of total CO2 per month and per transport mode

| year | month | transportMode | totalCO2_kg | avgCO2_kg |
|------|-------|---------------|-------------|-----------|
| 2022 | 1     | "BRT"         | 17.83       | 1.11      |
| 2022 | 1     | "LRT"         | 10.25       | 0.51      |
| 2022 | 1     | "Multimodal"  | 28.65       | 0.99      |
| 2022 | 1     | "Shuttle"     | 13.78       | 1.53      |
| 2022 | 2     | "BRT"         | 19.72       | 0.79      |
| 2022 | 2     | "LRT"         | 10.37       | 0.8       |
| 2022 | 2     | "Multimodal"  | 44.82       | 1.09      |
| 2022 | 2     | "Shuttle"     | 6.33        | 1.05      |
| 2022 | 3     | "BRT"         | 16.91       | 0.99      |
| 2022 | 3     | "LRT"         | 7.89        | 0.72      |

For lowest carbon intensity per km

| transportMode | co2_per_km |
|---------------|------------|
| "LRT"         | 18.24      |

Ar07: Which route/stop combination has the most unresolved High or Critical feedback in the last 6 months? List the top 5 along with common tags?

Cypher:

```
1 MATCH (f:Feedback)-[:ABOUT_ROUTE]->(r:Route)
2 MATCH (f)-[:ABOUT_STOP]->(s:Stop)
3 WITH
4     f, r, s,
5     datetime(f.submitted_at) AS dt
6 WHERE f.is_resolved = false
7     AND f.severity IN ["High", "Critical"]
8     AND dt >= datetime("2024-12-30T16:32:08") - duration("P6M")
9
10 WITH
11     r.route_id AS route,
12     s.stop_name AS stop,
13     collect(f.tags) AS allTags,
14     count(f) AS unresolvedCount
15
16 RETURN
17     route,
18     stop,
19     unresolvedCount,
20     allTags
21 ORDER BY unresolvedCount DESC
```

Result:

|   | route    | stop                    | unresolvedCount | allTags                                                                                                   |
|---|----------|-------------------------|-----------------|-----------------------------------------------------------------------------------------------------------|
| 1 | "BRT-02" | "Rawang Micro-Hub"      | 1               | [[{"tag": "ticketing", "count": 1}, {"tag": "app-bug", "count": 1}, {"tag": "peak-hour", "count": 1}]]    |
| 2 | "BRT-04" | "Yong Peng LRT Station" | 1               | [[{"tag": "praise", "count": 1}, {"tag": "bike-damage", "count": 1}]]                                     |
| 3 | "BRT-05" | "Serdang Bus Stop"      | 1               | [[{"tag": "first-mile", "count": 1}, {"tag": "last-mile", "count": 1}, {"tag": "peak-hour", "count": 1}]] |
| 4 | "BRT-05" | "Stadium Interchange"   | 1               | [[{"tag": "overcrowded", "count": 1}, {"tag": "peak-hour", "count": 1}]]                                  |
| 5 | "BRT-02" | "West Market Station"   | 1               | [[{"tag": "peak-hour", "count": 1}, {"tag": "delay", "count": 1}]]                                        |

Ar08: For Gold and Platinum tier passengers, what is the average number of journeys per month, preferred transport mode, and average rating given?

Cypher:

```

1 // Group by loyalty tier with NaN ratings treated as 0
2 MATCH (p:Passenger)-[:MADE]->(j:Journey)
3 WHERE p.loyalty_tier IN ['Gold', 'Platinum']
4 WITH p,
5     LEFT(j.journey_date, 7) AS year_month,
6     j.transport_mode AS mode,
7     CASE
8         WHEN j.rating IS NULL OR toString(j.rating) = 'NaN' THEN 0.0
9         ELSE j.rating
10    END AS rating
11 // Per passenger per month
12 WITH p, year_month, mode,
13     COUNT(*) AS monthly_journeys,
14     AVG(rating) AS monthly_rating
15 // Per passenger
16 WITH p,
17     AVG(monthly_journeys) AS avg_monthly,
18     AVG(monthly_rating) AS avg_rating,
19     COLLECT(DISTINCT mode) AS modes_used
20 // Per passenger - find preferred mode
21 WITH p,
22     avg_rating,
23     [m IN modes_used WHERE m IS NOT NULL AND m <> ""][0] AS preferred_mode
24 // Group by loyalty_tier
25 WITH p.loyalty_tier AS loyalty_tier,
26     ROUND(AVG(avg_monthly), 2) AS avg_journeys_per_month,
27     COLLECT(preferred_mode) AS all_modes,
28     ROUND(AVG(avg_rating), 2) AS avg_rating_given
29 WITH loyalty_tier,
30     avg_journeys_per_month,
31     avg_rating_given,
32     all_modes,
33     // Find most common preferred mode
34     REDUCE(best = {mode: "", count: 0}, m IN all_modes |
35         CASE
36             WHEN m IS NOT NULL AND m <> "" AND SIZE([x IN all_modes WHERE x = m]) > best.count
37             THEN {mode: m, count: SIZE([x IN all_modes WHERE x = m])}
38             ELSE best
39         END
40     ) AS top_mode
41 RETURN loyalty_tier,
42     avg_journeys_per_month,
43     top_mode.mode AS preferred_transport_mode,
44     avg_rating_given
45 ORDER BY loyalty_tier

```

Result:

| loyalty_tier | avg_journeys_per_month | preferred_transport_mode | avg_rating_given |
|--------------|------------------------|--------------------------|------------------|
| "Gold"       | 1.0                    | "Multimodal"             | 1.67             |
| "Platinum"   | 1.02                   | "Multimodal"             | 1.97             |

Ar09: Using the interchange connection data in the database, find all stops reachable from Central Interchange (STOP-001) within 2 transfer hops. For each reachable stop, return its name, zone, and the routes available at that stop.

Cypher:

```

1 MATCH (start:Stop {stop_id: "STOP-001"})
2 CALL {
3   WITH start
4   // 1 hop
5   MATCH (start)-[:SERVES]-(r:Route)-[:SERVES]->(stop1:Stop)
6   WHERE stop1 <> start
7   RETURN DISTINCT stop1 AS stop, r AS route, 1 AS hops
8
9   UNION
10
11  // 2 hops
12  MATCH (start)-[:SERVES]-(r1:Route)-[:SERVES]->(mid:Stop)-[:SERVES]-(r2:Route)-[:SERVES]->
    (stop2:Stop)
13  WHERE mid <> start
14  AND stop2 <> start
15  AND stop2 <> mid
16  RETURN DISTINCT stop2 AS stop, r2 AS route, 2 AS hops
17 }
18 WITH stop, hops, COLLECT(DISTINCT route) AS routes
19 RETURN
20     stop.stop_name AS stop_name,
21     stop.zone AS zone,
22     [r IN routes | r.route_id] AS route_ids,
23     [r IN routes | r.route_name] AS route_names
24 ORDER BY hops, stop.stop_name

```

Result:

Part of result

| stop_name                | zone     | route_ids           | route_names                                                              |
|--------------------------|----------|---------------------|--------------------------------------------------------------------------|
| "Airport Terminal Hub"   | "Zone F" | ["LRT-A"]           | ["Green Line – Airport to Northgate"]                                    |
| "Ampang Pecah Micro-Hub" | "Zone C" | ["BRT-11"]          | ["Medical Hub – Central Interchange"]                                    |
| "Bahau Bus Stop"         | "Zone F" | ["BRT-06", "LRT-A"] | ["Central Interchange – Tech Park", "Green Line – Airport to Northgate"] |
| "Bangi Micro-Hub"        | "Zone E" | ["BRT-01"]          | ["City Centre – Northgate Express"]                                      |
| "Buloh Kasap Bus Stop"   | "Zone F" | ["LRT-A"]           | ["Green Line – Airport to Northgate"]                                    |
| "Cheras LRT Station"     | "Zone F" | ["LRT-B"]           | ["Blue Line – East-West Crossstown"]                                     |

Ar10: Identify passengers who have more than 20% of their journeys with status 'Cancelled' or 'Incomplete' in 2024, along with their total journey count and home zone.  
Cypher:

```
1 MATCH (p:Passenger)-[:MADE]->(j:Journey)
2 WHERE j.journey_date STARTS WITH '2024'
3 WITH p,
4     COUNT(j) AS total_journeys,
5     SUM(CASE WHEN j.status IN ['Cancelled', 'Incomplete'] THEN 1 ELSE 0 END) AS bad_journeys
6 WHERE total_journeys > 0 // Avoid division by zero
7     AND (bad_journeys * 1.0 / total_journeys) > 0.20
8 RETURN p.passenger_id AS passenger_id,
9        p.full_name AS full_name,
10       p.home_zone AS home_zone,
11       total_journeys,
12       bad_journeys AS cancelled_incomplete_count,
13       ROUND((bad_journeys * 100.0 / total_journeys), 2) AS bad_percentage
14 ORDER BY bad_percentage DESC
```

Result:  
Part of result

| passenger_id                             | full_name               | home_zone | total_journeys | cancelled_incomplete | bad_percentage |
|------------------------------------------|-------------------------|-----------|----------------|----------------------|----------------|
| 1 "fcb1a281-2d53-4810-bc01-c46c54ac13ce" | "Ms Hilary Walte rs"    | "Zone A"  | 2              | 2                    | 100.0          |
| 2 "702805af-5806-41f1-b820-242631341ab8" | "Joel Stephens"         | "Zone B"  | 2              | 2                    | 100.0          |
| 3 "5fc57ff4-d7a9-48ea-af7c-db39283edeec" | "Terry Smith-Har rison" | "Zone D"  | 2              | 2                    | 100.0          |
| 4 "f20ba0c9-2a86-4922-b492-7b3536c21bc3" | "Lydia Bishop"          | "Zone D"  | 1              | 1                    | 100.0          |
| 5 "970e287a-0fd7-40f2-8f90-8490006e0070" | "Patrick Jones"         | "Zone F"  | 2              | 2                    | 100.0          |



## Appendix C: Database Backup Verification Screenshots

```

1 export.json 1 X
2 D:\1.UTM > Course > Semester4 > No DB SQL > Asg > mini project > Submitted file > Database Backup file > 1 export.json > ...
3 1 [{"type":"node","id":"0","labels":["Journey"],"properties":{"distance_km":17.81,"passenger_id":"cadac122-0374-4864-9981-d28fa09c4325","carbon_b":616.36,"rating":"NaN"},"pr
4 2 [{"type":"node","id":"1","labels":["Journey"],"properties":{"distance_km":10.86,"passenger_id":"f35ba0b20-4d28-48c8-9f5e-f823493a39cf","carbon_b":398.82,"rating":"NaN"},"pr
5 3 [{"type":"node","id":"2","labels":["Journey"],"properties":{"distance_km":18.25,"passenger_id":"9d2c5e1b-37e8-4d71-a6e6-a2743ca80d97","carbon_b":328.92,"rating":5.0,"prom
6 4 [{"type":"node","id":"3","labels":["Journey"],"properties":{"distance_km":19.06,"passenger_id":"f689ca42-2ca2-4887-b161-68c4a3188a65","carbon_b":1170.03,"rating":"NaN"},"pr
7 5 [{"type":"node","id":"4","labels":["Journey"],"properties":{"distance_km":9.66,"passenger_id":"50c4c4b4-ad9c-4b97-40d9a90a41b8","carbon_b":482.37,"rating":"NaN"},"pr
8 6 [{"type":"node","id":"5","labels":["Journey"],"properties":{"distance_km":21.62,"passenger_id":"ade18084-e1f9-4413-bfda-66661313421","carbon_b":727.39,"rating":5.0,"pro
9 7 [{"type":"node","id":"6","labels":["Journey"],"properties":{"distance_km":7.99,"passenger_id":"a2738481-6176-4f0e-41be-91a742007f95","carbon_b":324.71,"rating":"NaN"},"pr
10 8 [{"type":"node","id":"7","labels":["Journey"],"properties":{"distance_km":13.07,"passenger_id":"3ff09d41-df47-5599-af9e-dd4858642597","carbon_b":161.74,"rating":1.0,"pro
11 9 [{"type":"node","id":"8","labels":["Journey"],"properties":{"distance_km":10.48,"passenger_id":"850b7073-7f13-4e67-9008-cde99d421bae","carbon_b":422.2,"rating":5.0,"prom
12 10 [{"type":"node","id":"9","labels":["Journey"],"properties":{"distance_km":22.22,"passenger_id":"0296435d-b0b6-45da-a8c1-8df045ca7fa5","carbon_b":1514.26,"rating":4.0,"pr
13 11 [{"type":"node","id":"10","labels":["Journey"],"properties":{"distance_km":15.63,"passenger_id":"cbb1d1ce-470b-46a2-8363-0f7885f8da55","carbon_b":452.91,"rating":"NaN"},"pr
14 12 [{"type":"node","id":"11","labels":["Journey"],"properties":{"distance_km":13.24,"passenger_id":"89344b06-0790-46ba-136a-9761315e4e80","carbon_b":371.46,"rating":"NaN"},"pr
15 13 [{"type":"node","id":"12","labels":["Journey"],"properties":{"distance_km":11.9,"passenger_id":"b0d8fa76-1f16-4e17-8f9f-d562dd43b0cf","carbon_b":493.59,"rating":"NaN"},"pr
16 14 [{"type":"node","id":"13","labels":["Journey"],"properties":{"distance_km":15.37,"passenger_id":"512393fa-a8a1-432f-8f81-62eb4562624d","carbon_b":335.01,"rating":"NaN"},"pr
17 15 [{"type":"node","id":"14","labels":["Journey"],"properties":{"distance_km":10.07,"passenger_id":"35b806da-864f-46dd-a9ec-09d858ced9d9","carbon_b":416.84,"rating":"NaN"},"pr
18 16 [{"type":"node","id":"15","labels":["Journey"],"properties":{"distance_km":14.76,"passenger_id":"7f4c89fe-5fe0-47c3-4336-b60eb95f71db","carbon_b":920.52,"rating":"NaN"},"pr
19 17 [{"type":"node","id":"16","labels":["Journey"],"properties":{"distance_km":22.62,"passenger_id":"07a25155-65f7-4d06-9a2d-465b87cebaaa","carbon_b":994.75,"rating":3.0,"pr
20 18 [{"type":"node","id":"17","labels":["Journey"],"properties":{"distance_km":16.0,"passenger_id":"d320878e-43a8-43a9-94a5-587ef6f88f1","carbon_b":383.84,"rating":"NaN"},"pr
21 19 [{"type":"node","id":"18","labels":["Journey"],"properties":{"distance_km":34.75,"passenger_id":"57b0dbac-854c-40ea-b9d9-92cd8db08b8b","carbon_b":1186.05,"rating":5.0,"pr
22 20 [{"type":"node","id":"19","labels":["Journey"],"properties":{"distance_km":23.39,"passenger_id":"075bd148-71ab-4b71-a279-4a6e61f14c98","carbon_b":535.35,"rating":"NaN"},"pr
23 21 [{"type":"node","id":"20","labels":["Journey"],"properties":{"distance_km":6.9,"passenger_id":"07aa2515-65f7-4d06-9a2d-465b87cebaaa","carbon_b":197.21,"rating":4.0,"prom
24 22 [{"type":"node","id":"21","labels":["Journey"],"properties":{"distance_km":11.16,"passenger_id":"393377aa-9c4e-4b62-8068-49cd2dbfbc31","carbon_b":573.12,"rating":"NaN"},"pr
25 23 [{"type":"node","id":"22","labels":["Journey"],"properties":{"distance_km":20.93,"passenger_id":"29decbcc-429e-49b8-4081-d562409c2e4ab7","carbon_b":696.18,"rating":4.0,"pr
26 24 [{"type":"node","id":"23","labels":["Journey"],"properties":{"distance_km":13.97,"passenger_id":"12b3c5eb-419d-4029-98e1-b84c-881080cfcaa","carbon_b":419.05,"rating":"NaN"},"pr
27 25 [{"type":"node","id":"24","labels":["Journey"],"properties":{"distance_km":5.92,"passenger_id":"7b163453-7042-4b9e-8612-85a7b6f9991","carbon_b":235.61,"rating":"NaN"},"pr
28 26 [{"type":"node","id":"25","labels":["Journey"],"properties":{"distance_km":11.7,"passenger_id":"633bf0ff-40de-bcf-8630-555b5c7b79a1","carbon_b":634.95,"rating":"NaN"},"pr
29 27 [{"type":"node","id":"26","labels":["Journey"],"properties":{"distance_km":26.97,"passenger_id":"c3c5bb05-e355-4e16-954a-7ea03354abde","carbon_b":748.65,"rating":"NaN"},"pr
30 28 [{"type":"node","id":"27","labels":["Journey"],"properties":{"distance_km":24.88,"passenger_id":"845c10ea-1084-4e8f-ac02-e881080cfcaa","carbon_b":994.84,"rating":3.0,"pr
31 29 [{"type":"node","id":"28","labels":["Journey"],"properties":{"distance_km":8.65,"passenger_id":"16054ab9-313a-4c74-8a3d-b769cf952461","carbon_b":331.97,"rating":"NaN"},"pr
32 30 [{"type":"node","id":"29","labels":["Journey"],"properties":{"distance_km":17.72,"passenger_id":"b55beaa4-c5b3-404b-9e7b-4866d2705470","carbon_b":1289.26,"rating":4.0,"pr
33 31 [{"type":"node","id":"30","labels":["Journey"],"properties":{"distance_km":12.98,"passenger_id":"f8efc7f1-2b9f-4f6a-a666-499459364dc","carbon_b":767.41,"rating":4.0,"pr
34 32 [{"type":"node","id":"31","labels":["Journey"],"properties":{"distance_km":13.67,"passenger_id":"12249735-f254-4c17-af90-ca8862a3a484","carbon_b":523.6,"rating":"NaN"},"pr
35 33 [{"type":"node","id":"32","labels":["Journey"],"properties":{"distance_km":20.44,"passenger_id":"48cd42c0-5a66-419f-95f-804660c1d180","carbon_b":516.74,"rating":"NaN"},"pr
36 34 [{"type":"node","id":"33","labels":["Journey"],"properties":{"distance_km":18.43,"passenger_id":"12101787-3429-4a38-8931-669724abab28","carbon_b":463.76,"rating":4.0,"pr
37 35 [{"type":"node","id":"34","labels":["Journey"],"properties":{"distance_km":6.5,"passenger_id":"4cd7e6ff-432e-4d2c-47a6-16d00a05d86","carbon_b":218.94,"rating":"NaN"},"pr
38 36 [{"type":"node","id":"35","labels":["Journey"],"properties":{"distance_km":17.1,"passenger_id":"2f4c89f0-54e9-47c3-a366-60d90591916","carbon_b":921.69,"rating":5.0,"prom

```